

home_examv1

April 18, 2025

1 Deep Learning Project Development Summary (FYS-3033)

This document summarizes the key development steps, architectural design choices, and clever improvements made during the implementation of the deep learning classifier for the home exam in FYS-3033. This is a behind-the-scenes overview of the insights and decisions made to refine the workflow, training pipeline, and model performance.

1.1 General Improvements and Workflow Design

1.1.1 Modular Code Structure

- **Split into src/ submodules:**
 - `vgg11bn.py`: Manually implemented VGG-11 with BatchNorm & optional Dropout.
 - `train.py`: Self-contained training loop with live tqdm feedback, logging, model saving.
 - `utils.py`: Contains custom Dataset, entropy function, seed function, etc.
 - `evaluation.py`: Confusion matrix, classification report, ROC/entropy plotting.

1.1.2 Project Tree Setup

- Ensured clean file/folder separation using `.gitignore`, `plots/`, `logs/`, `models/`, and `doc/` folders.
- Included `.gitkeep` files for empty folders to retain structure in git.
- Used `show_tree.py` to quickly inspect and visualize directory contents.

1.1.3 Debugging and Reproducibility

- Created `set_seed()` function to fix random seeds for PyTorch, NumPy, and CUDA.
 - Identified issues with `__pycache__` and `*.ipynb_checkpoints` interfering with Git commits.
 - Learned how to clean up git merge/commit issues using `rm .git/index.lock` and reset commands.
-

1.2 Model Architecture (VGG-11 BN)

1.2.1 Design Justifications

- Started with VGG-11 (column A in Simonyan & Zisserman) + added BatchNorm after every Conv.

- Made Dropout optional using:

```
nn.Dropout(p=0.5) if self.dropout else nn.Identity()
```

This smart use of `nn.Identity()` ensured flexible model instantiation.

1.2.2 Improvements Made

- Switched `stride=1` and `padding=1` in Conv layers to preserve spatial resolution.
- Used `.view(x.size(0), -1)` instead of `.flatten()` for safety and PyTorch consistency.

1.3 Training Pipeline Enhancements

1.3.1 Metrics Logging

- Tracked and stored `train_loss`, `val_loss`, `val_acc` into CSV files.
- Automatically plotted curves and saved them as `.png`.
- Example:

```
logs/VGG11BN_2c_with_dropout_metrics.csv
plots/VGG11BN_2c_with_dropout_training.png
```

1.3.2 Live Feedback with tqdm

- Swapped out standard loops with `tqdm()` and live metric updates:

```
loop.set_postfix(loss=f"{loss.item():.4f}", acc=f"{batch_acc:.4f}")
```

1.3.3 Model Evaluation

- Implemented:
 - `classification_summary()` with support for returning final val loss and acc
 - `plot_confusion_matrix()` using `sklearn`
 - `plot_multiclass_roc()` for ROC curve analysis per class
 - `print_framed_metrics()` to display results in clean framed output.

1.4 Experiments Conducted

1.4.1 Config Table

Model Name	Dropout	Weight Decay	Scheduler
2a - Baseline	No	No	No
2a - +WD	No	1e-4	No
2a - +WD+Sched	No	1e-4	Yes
2c - Dropout	Yes	No	No
2c - +WD	Yes	1e-4	No
2c - +WD+Sched	Yes	1e-4	Yes

1.4.2 Results (example summary)

Model	Val Loss	Val Acc	
64	2a	0.9112	0.8900
2c	0.7322	0.8833	

1.5 Git & Notebook Handling

- Avoided tracking notebook-generated artifacts (`__pycache__`, checkpoints, etc.)
 - Pushed Jupyter results only when needed.
 - Used `git rm --cached file` and `.gitignore` to keep the repo clean.
-

1.6 General Takeaways

- Dropout helped lower validation loss (better calibration), despite small drop in accuracy.
 - Weight decay + learning rate scheduler stabilized training.
 - Batch size was constant (32), but future work may test 64/128 for throughput vs. stability.
 - All model performance metrics are reproducible and properly logged/saved.
-

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import sys
import os

# Add the src folder to the Python path
sys.path.append(os.path.join(os.getcwd(), 'src/'))

training_data = np.load('data/problem2/training_data.npz')
training_images = training_data['a']
training_labels = training_data['b']

print('training_images shape:', training_images.shape)
print('training_labels shape:', training_labels.shape)
# converting to dataframe
df = pd.DataFrame({
    'images': list(training_images), # Convert numpy array to list for DataFrame
    'labels': training_labels # Keep labels as is
})

# Display the first few rows of the DataFrame
print(df.head())

import matplotlib.gridspec as gridspec
from collections import Counter
```

```

counter = Counter(training_labels)

fig = plt.figure(figsize=(20, 10))
gs = gridspec.GridSpec(2, 1, height_ratios=[1, 1.2])

# Top row: display 5 sample images in a row
gs_top = gridspec.GridSpecFromSubplotSpec(1, 5, subplot_spec=gs[0])
for i in range(5):
    ax = fig.add_subplot(gs_top[i])
    img = training_images[i].transpose(1, 2, 0) # (C, H, W) -> (H, W, C)
    ax.imshow(img)
    ax.set_title(f"Label: {training_labels[i]}")
    ax.axis('off')

# Bottom row: bar chart for the class distribution
ax_bar = fig.add_subplot(gs[1])
ax_bar.bar(list(counter.keys()), list(counter.values()))
ax_bar.set_title("Class Distribution in Training Set")
ax_bar.set_xticks([0, 1, 2])
ax_bar.set_xticklabels(['Plane', 'Ship', 'Truck'])

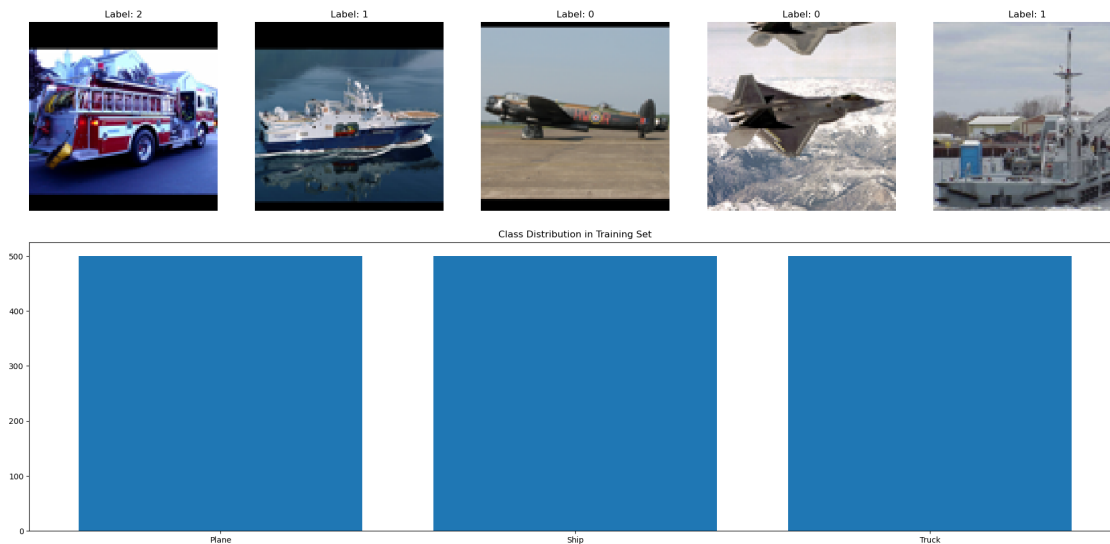
plt.tight_layout()
plt.show()

```

training_images shape: (1500, 3, 96, 96)

training_labels shape: (1500,)

	images	labels
0	[[[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0...	2
1	[[[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0...	1
2	[[[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0...	0
3	[[[218, 216, 216, 216, 218, 218, 216, 217, 220...	0
4	[[[181, 178, 177, 181, 184, 184, 185, 187, 190...	1



```
[2]: from sklearn.model_selection import train_test_split
```

```
# Split indices
train_idx, val_idx = train_test_split(
    np.arange(len(training_labels)),
    test_size=0.2,
    stratify=training_labels,
    random_state=42
)

# Create sets
train_images = training_images[train_idx]
train_labels = training_labels[train_idx]
val_images = training_images[val_idx]
val_labels = training_labels[val_idx]
```

```
[3]: # from torch.utils.data import DataLoader
# from src.utils import ImageDataset

# # Create datasets
# train_dataset = ImageDataset(train_images, train_labels)
# val_dataset = ImageDataset(val_images, val_labels)

# # Create loaders
# dataloaders = {
#     'train': DataLoader(
#         train_dataset,
```

```

#         batch_size=32,
#         shuffle=True,           # Shuffles training data each epoch
#         pin_memory=True        # Speeds up transfer to GPU (if using CUDA)
#     ),
#     'val': DataLoader(
#         val_dataset,
#         batch_size=32,
#         shuffle=False,         # Keeps validation consistent across epochs
#         pin_memory=True        # Still helpful for val on GPU
#     )
# }

```

```

[4]: from torch.utils.data import DataLoader
from src.utils import ImageDataset, set_seed, print_framed_metrics
from src.vgg11bn import VGG11BN
from src.train import train_model
from src.evaluation import classification_summary, plot_confusion_matrix,
    plot_multiclass_roc
import torch
import torch.nn as nn
import torch.optim as optim
from torch.optim.lr_scheduler import StepLR

# Set seed and device
set_seed(42)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# Prepare datasets once
train_dataset = ImageDataset(train_images, train_labels)
val_dataset = ImageDataset(val_images, val_labels)

# Define experiment configurations
configs = [
    {
        "name": "VGG11BN_2a_no_dropout",
        "dropout": False,
        "weight_decay": 0,
        "use_scheduler": False,
        "batch_size": 32
    },
    {
        "name": "VGG11BN_2a_no_dropout_weightdecay",
        "dropout": False,
        "weight_decay": 1e-4,
        "use_scheduler": False,
        "batch_size": 32
    },
]

```

```

{
  "name": "VGG11BN_2a_no_dropout_weightdecay_scheduler",
  "dropout": False,
  "weight_decay": 1e-4,
  "use_scheduler": True,
  "batch_size": 32
},
{
  "name": "VGG11BN_2c_with_dropout",
  "dropout": True,
  "weight_decay": 0,
  "use_scheduler": False,
  "batch_size": 32
},
{
  "name": "VGG11BN_2c_with_dropout_weightdecay",
  "dropout": True,
  "weight_decay": 1e-4,
  "use_scheduler": False,
  "batch_size": 32
},
{
  "name": "VGG11BN_2c_with_dropout_weightdecay_scheduler",
  "dropout": True,
  "weight_decay": 1e-4,
  "use_scheduler": True,
  "batch_size": 32
},
{
  "name": "VGG11BN_2a_no_dropout",
  "dropout": False,
  "weight_decay": 0,
  "use_scheduler": False,
  "batch_size": 64
},
{
  "name": "VGG11BN_2a_no_dropout_weightdecay",
  "dropout": False,
  "weight_decay": 1e-4,
  "use_scheduler": False,
  "batch_size": 64
},
{
  "name": "VGG11BN_2a_no_dropout_weightdecay_scheduler",
  "dropout": False,
  "weight_decay": 1e-4,
  "use_scheduler": True,

```

```

        "batch_size": 64
    },
    {
        "name": "VGG11BN_2c_with_dropout",
        "dropout": True,
        "weight_decay": 0,
        "use_scheduler": False,
        "batch_size": 64
    },
    {
        "name": "VGG11BN_2c_with_dropout_weightdecay",
        "dropout": True,
        "weight_decay": 1e-4,
        "use_scheduler": False,
        "batch_size": 64
    },
    {
        "name": "VGG11BN_2c_with_dropout_weightdecay_scheduler",
        "dropout": True,
        "weight_decay": 1e-4,
        "use_scheduler": True,
        "batch_size": 64
    },
]

# Run each experiment
for cfg in configs:
    print(f"\n\n=== Training: {cfg['name']} (Batch size: {cfg['batch_size']})\n")

    # Create dataloaders with specified batch size
    dataloaders = {
        'train': DataLoader(
            train_dataset,
            batch_size=cfg['batch_size'],
            shuffle=True,
            pin_memory=True
        ),
        'val': DataLoader(
            val_dataset,
            batch_size=cfg['batch_size'],
            shuffle=False,
            pin_memory=True
        )
    }

```

```

# Model, optimizer, scheduler
model = VGG11BN(num_classes=3, dropout=cfg["dropout"]).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=1e-4,
↪weight_decay=cfg["weight_decay"])
scheduler = StepLR(optimizer, step_size=5, gamma=0.5) if
↪cfg["use_scheduler"] else None

# Train
trained_model = train_model(
    model=model,
    dataloaders=dataloaders,
    criterion=criterion,
    optimizer=optimizer,
    scheduler=scheduler,
    device=device,
    num_epochs=60,
    model_name=f"{cfg['name']}_bs{cfg['batch_size']}" # reflects batch
↪size in filenames
)

# Evaluate
val_loss, val_acc, y_true, y_pred = classification_summary(
    model=trained_model,
    dataloader=dataloaders["val"],
    criterion=criterion,
    device=device,
    class_names=['Plane', 'Ship', 'Truck']
)

print_framed_metrics(val_loss, val_acc)

# Plots
plot_confusion_matrix(
    trained_model,
    dataloaders['val'],
    device,
    class_names=['Plane', 'Ship', 'Truck'],
    save_path=f"plots/{cfg['name']}_bs{cfg['batch_size']}_confusion_matrix.
↪png"
)

plot_multiclass_roc(
    model=trained_model,
    dataloader=dataloaders['val'],
    device=device,

```

```

num_classes=3,
class_names=['Plane', 'Ship', 'Truck'],
model_name=f"{cfg['name']}_bs{cfg['batch_size']}"
)

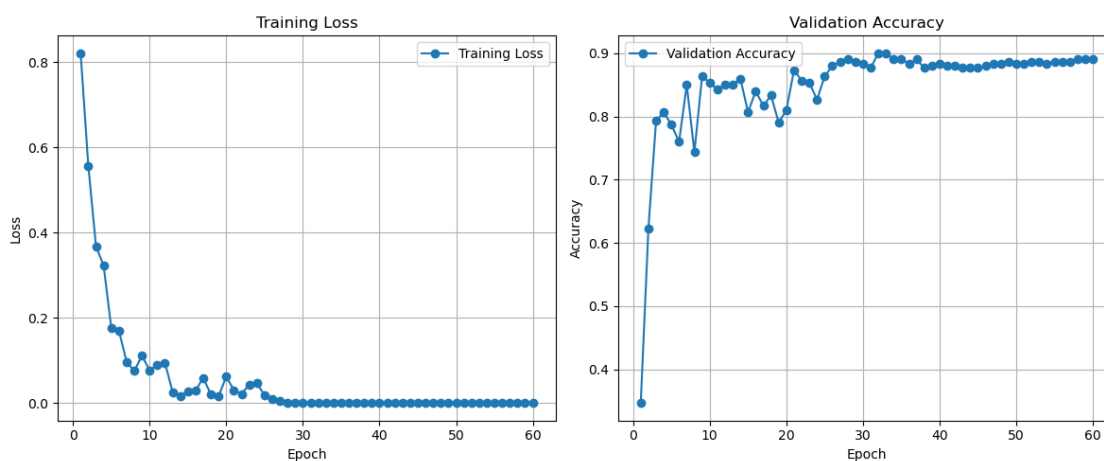
```

=== Training: VGG11BN_2a_no_dropout (Batch size: 32) ===

Training VGG11BN_2a_no_dropout_bs32_20250414_212007: 100%| | 2880/2880
[02:18<00:00, 20.75batch/s, epoch=60/60, phase=val, loss=1.3455, acc=0.9167]

Best Val Accuracy: 0.9000

Total training time: 138.77s

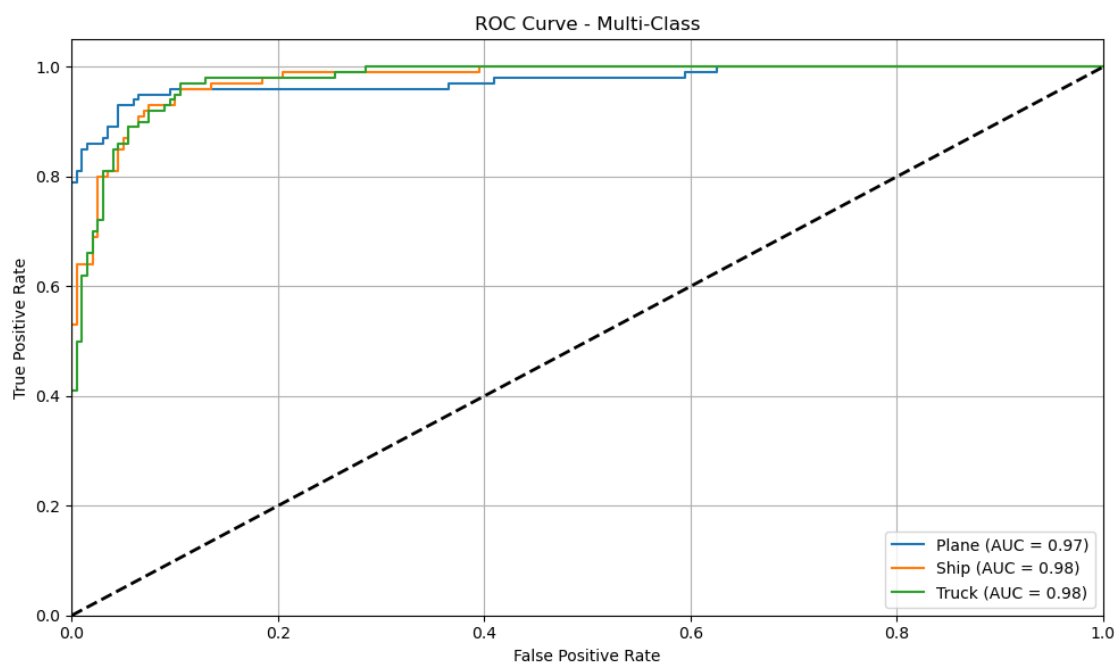
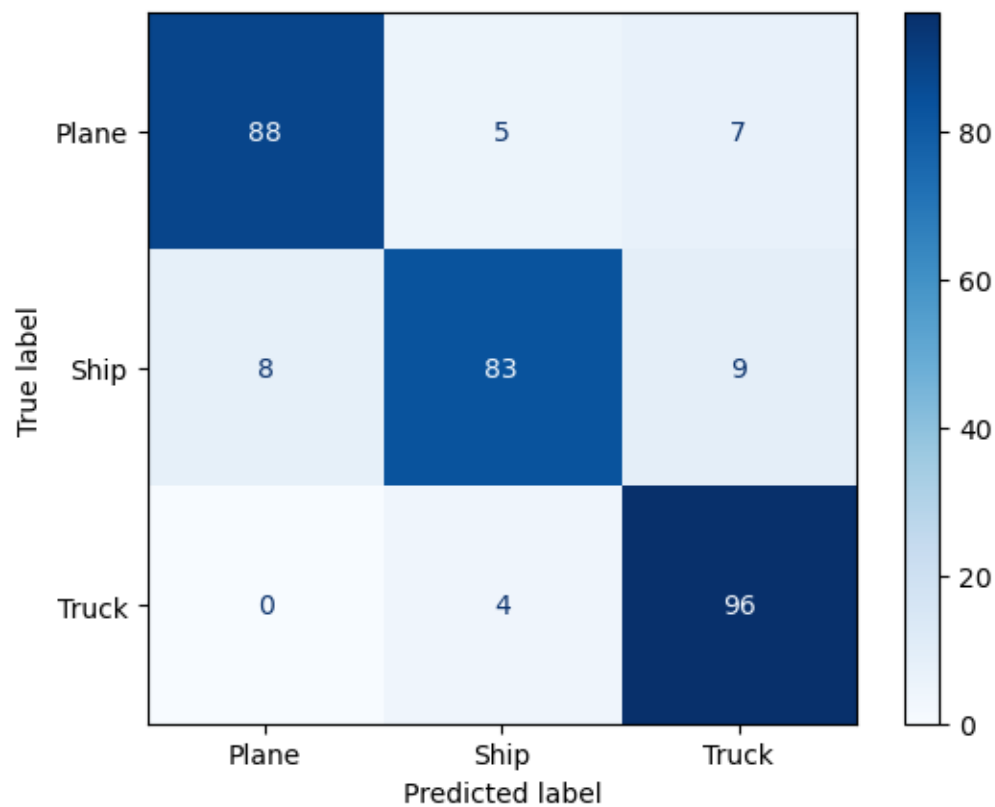


	precision	recall	f1-score	support
Plane	0.92	0.88	0.90	100
Ship	0.90	0.83	0.86	100
Truck	0.86	0.96	0.91	100
accuracy			0.89	300
macro avg	0.89	0.89	0.89	300
weighted avg	0.89	0.89	0.89	300

```

+-----+
| Validation Loss: 0.7407, Validation Accuracy: 0.8900 |
+-----+

```



[ROC] Saved ROC plot to: plots/ROC_VGG11BN_2a_no_dropout_bs32.png

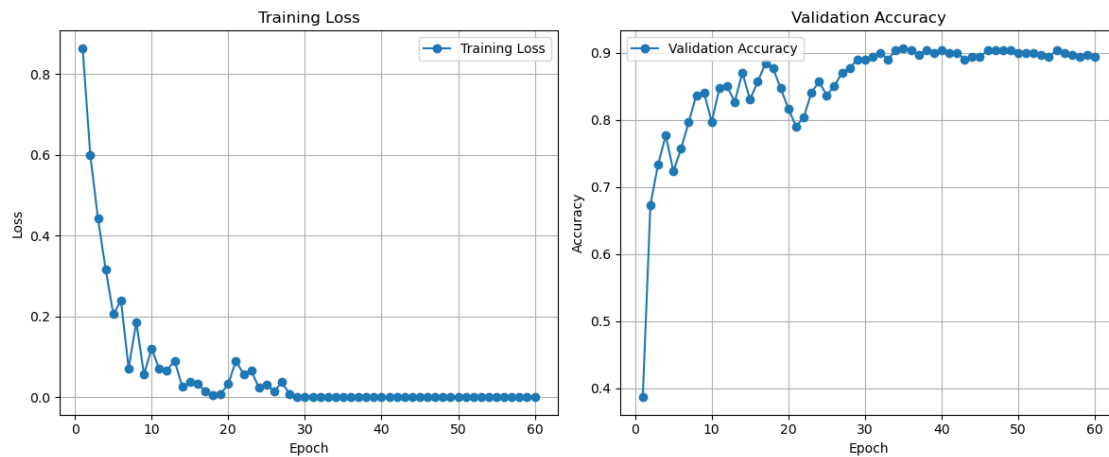
=== Training: VGG11BN_2a_no_dropout_weightdecay (Batch size: 32) ===

Training VGG11BN_2a_no_dropout_weightdecay_bs32_20250414_212228:

100%| | 2880/2880 [02:26<00:00, 19.67batch/s, epoch=60/60, phase=val,
loss=1.3797, acc=0.8333]

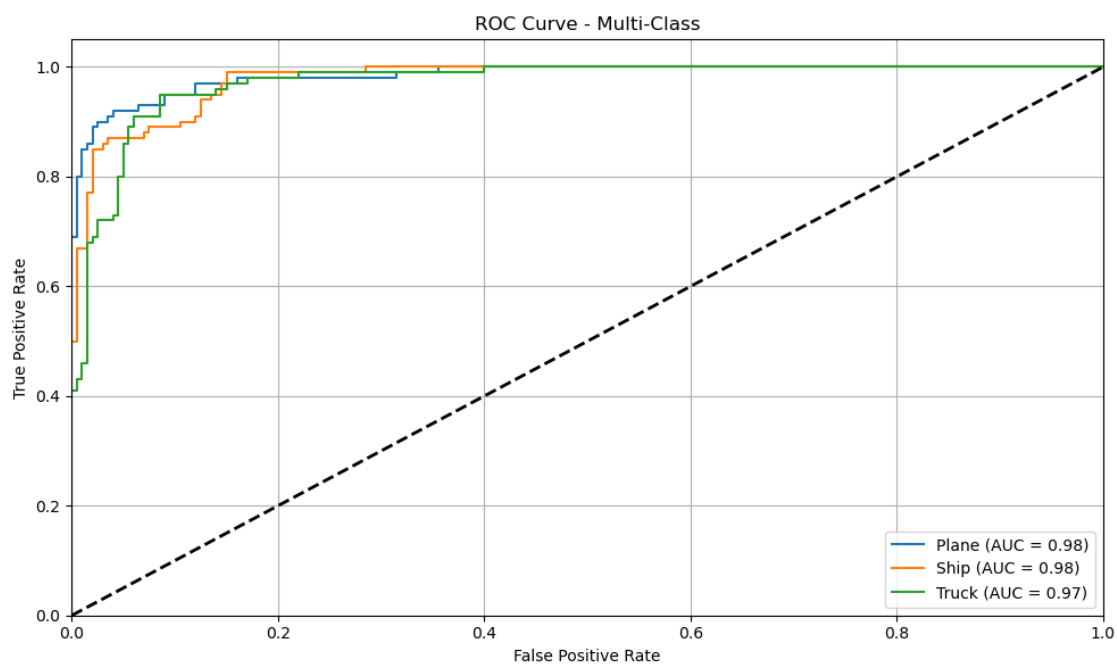
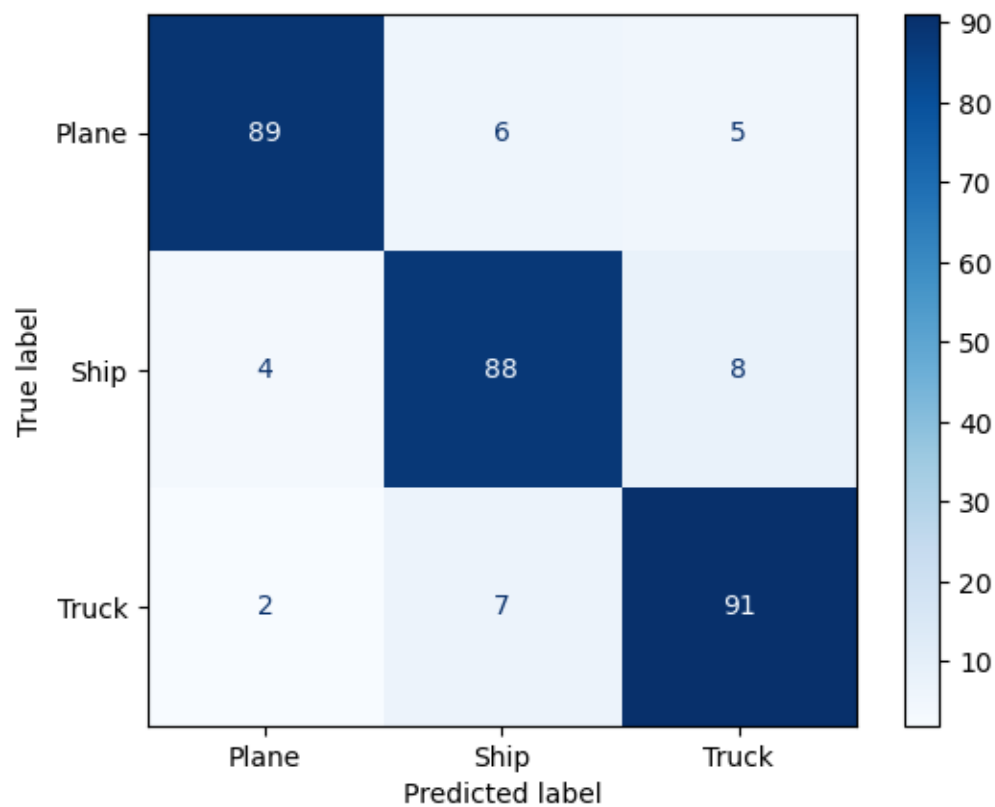
Best Val Accuracy: 0.9067

Total training time: 146.45s



	precision	recall	f1-score	support
Plane	0.94	0.89	0.91	100
Ship	0.87	0.88	0.88	100
Truck	0.88	0.91	0.89	100
accuracy			0.89	300
macro avg	0.89	0.89	0.89	300
weighted avg	0.89	0.89	0.89	300

```
+-----+
| Validation Loss: 0.5532, Validation Accuracy: 0.8933 |
+-----+
```



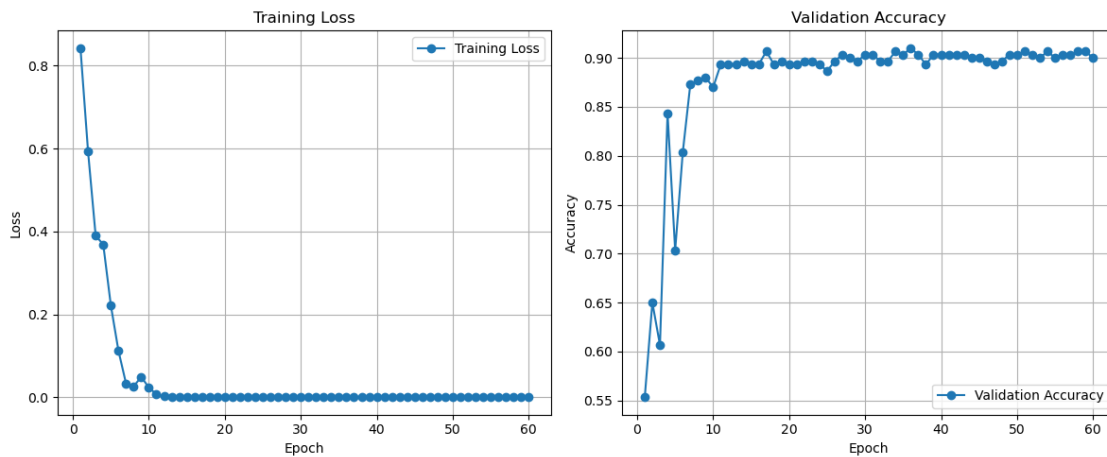
[ROC] Saved ROC plot to: plots/ROC_VGG11BN_2a_no_dropout_weightdecay_bs32.png

=== Training: VGG11BN_2a_no_dropout_weightdecay_scheduler (Batch size: 32) ===

Training VGG11BN_2a_no_dropout_weightdecay_scheduler_bs32_20250414_212456:
100%| | 2880/2880 [02:27<00:00, 19.49batch/s, epoch=60/60, phase=val,
loss=1.1399, acc=0.8333]

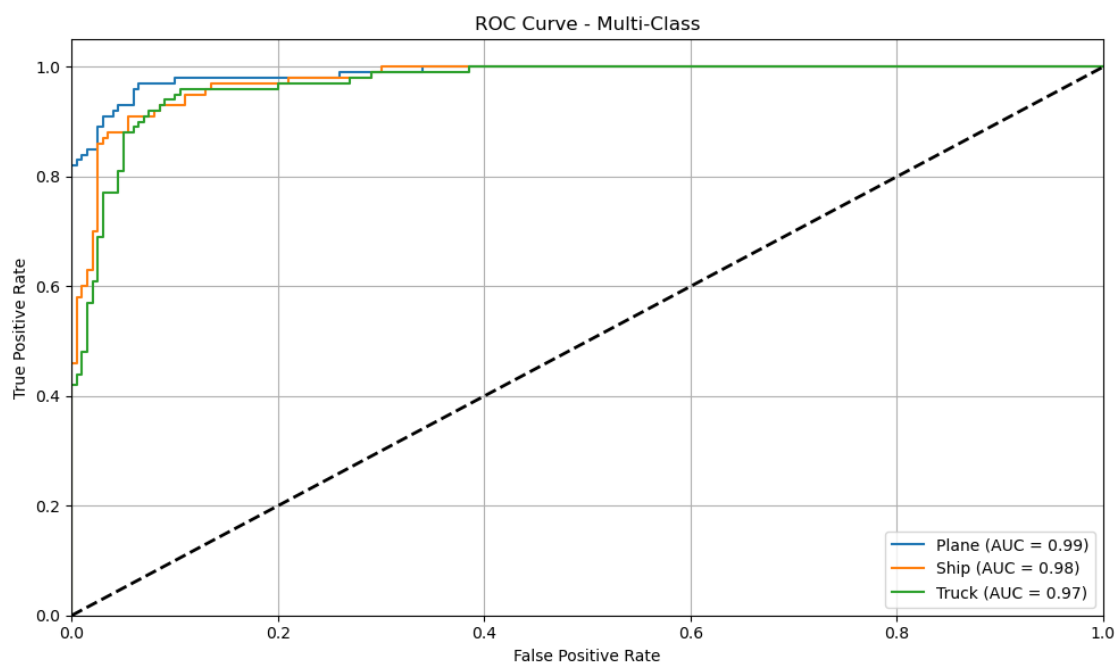
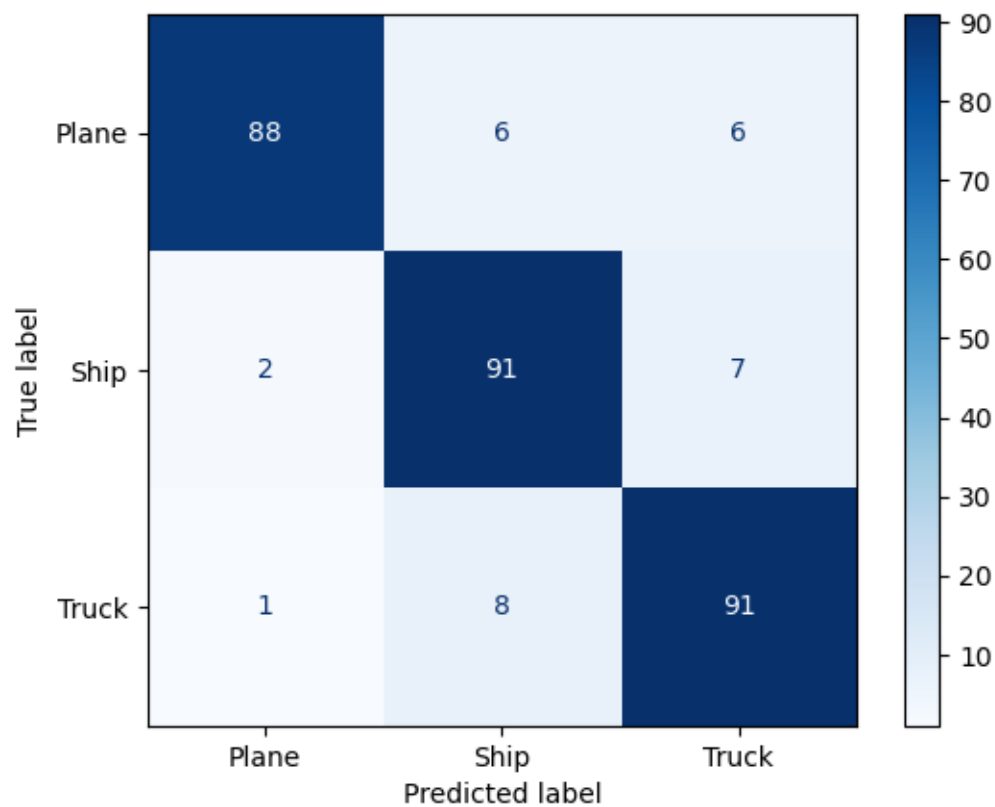
Best Val Accuracy: 0.9100

Total training time: 147.77s



	precision	recall	f1-score	support
Plane	0.97	0.88	0.92	100
Ship	0.87	0.91	0.89	100
Truck	0.88	0.91	0.89	100
accuracy			0.90	300
macro avg	0.90	0.90	0.90	300
weighted avg	0.90	0.90	0.90	300

```
+-----+
| Validation Loss: 0.4317, Validation Accuracy: 0.9000 |
+-----+
```



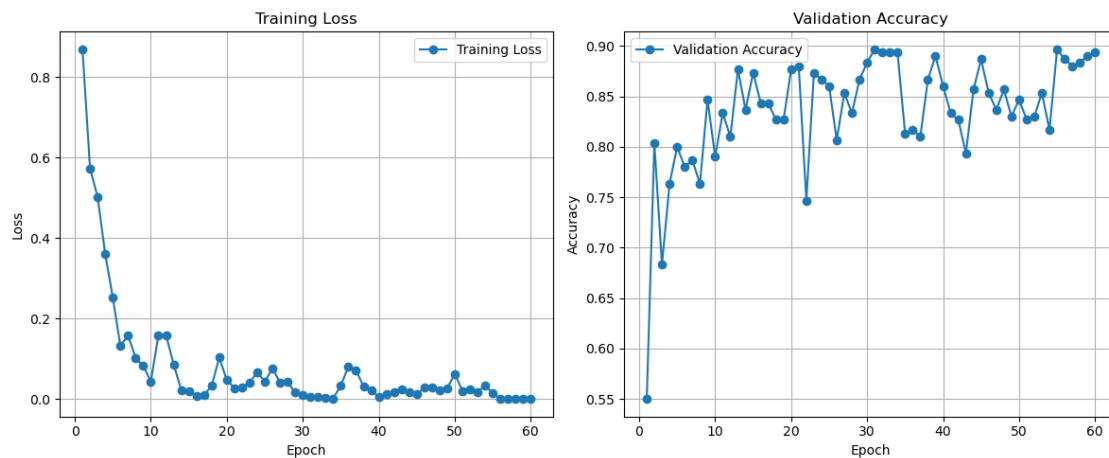
[ROC] Saved ROC plot to:
plots/ROC_VGG11BN_2a_no_dropout_weightdecay_scheduler_bs32.png

=== Training: VGG11BN_2c_with_dropout (Batch size: 32) ===

Training VGG11BN_2c_with_dropout_bs32_20250414_212726: 100%|
2880/2880 [02:23<00:00, 20.06batch/s, epoch=60/60, phase=val, loss=2.3188,
acc=0.8333]

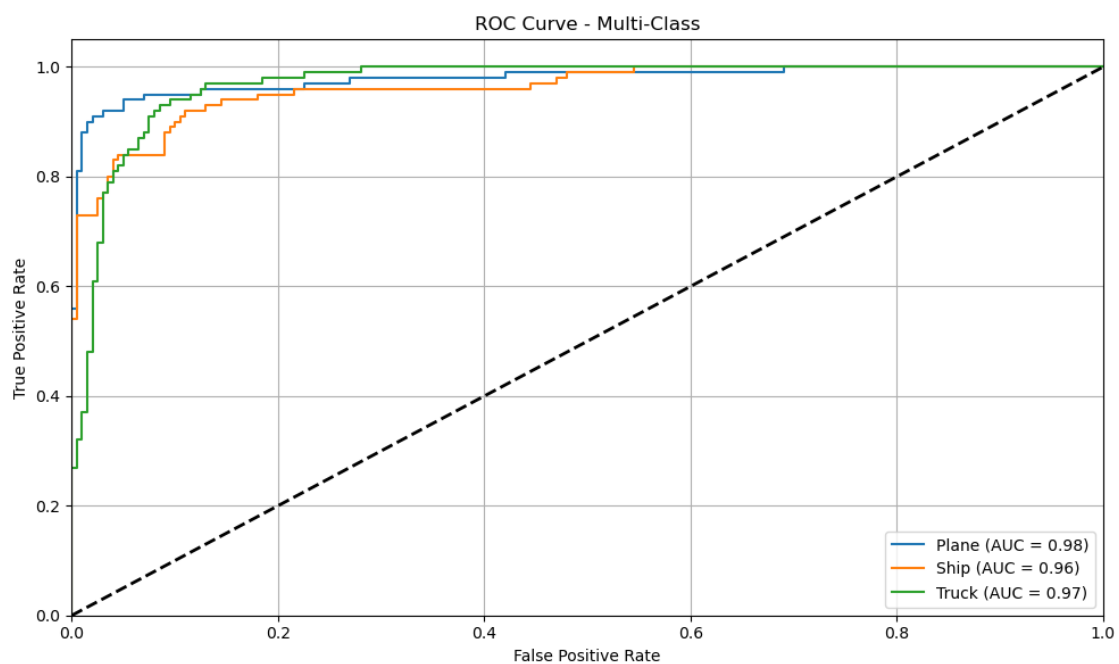
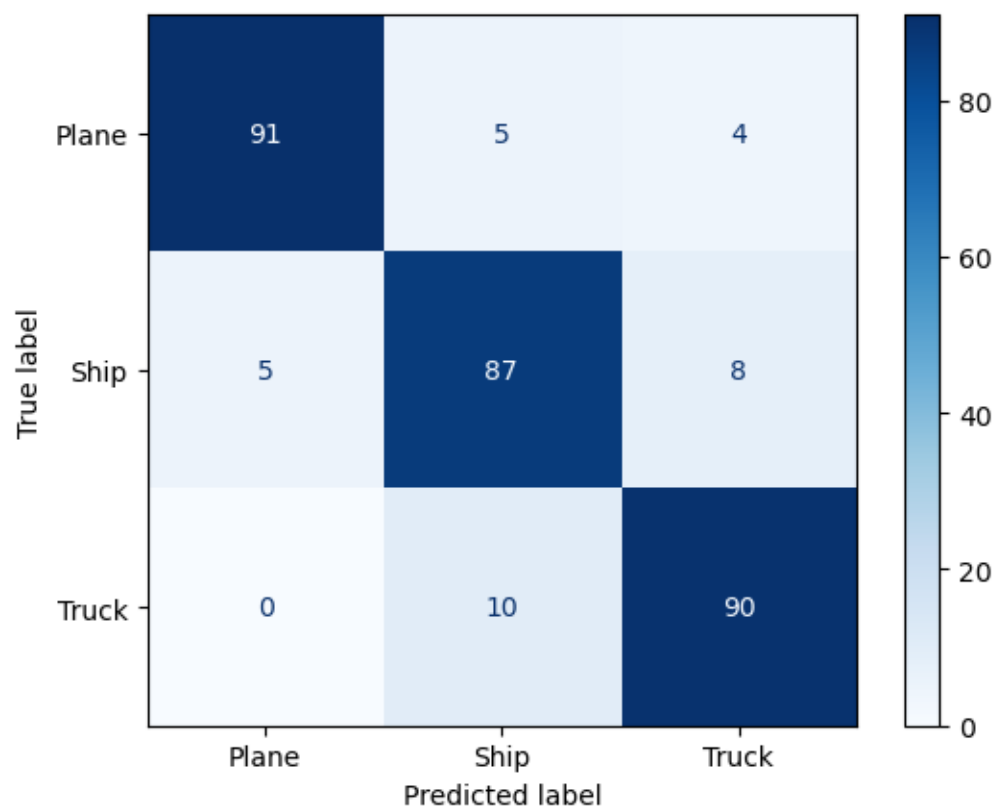
Best Val Accuracy: 0.8967

Total training time: 143.54s



	precision	recall	f1-score	support
Plane	0.95	0.91	0.93	100
Ship	0.85	0.87	0.86	100
Truck	0.88	0.90	0.89	100
accuracy			0.89	300
macro avg	0.89	0.89	0.89	300
weighted avg	0.89	0.89	0.89	300

+-----+
| Validation Loss: 0.6461, Validation Accuracy: 0.8933 |
+-----+



[ROC] Saved ROC plot to: plots/ROC_VGG11BN_2c_with_dropout_bs32.png

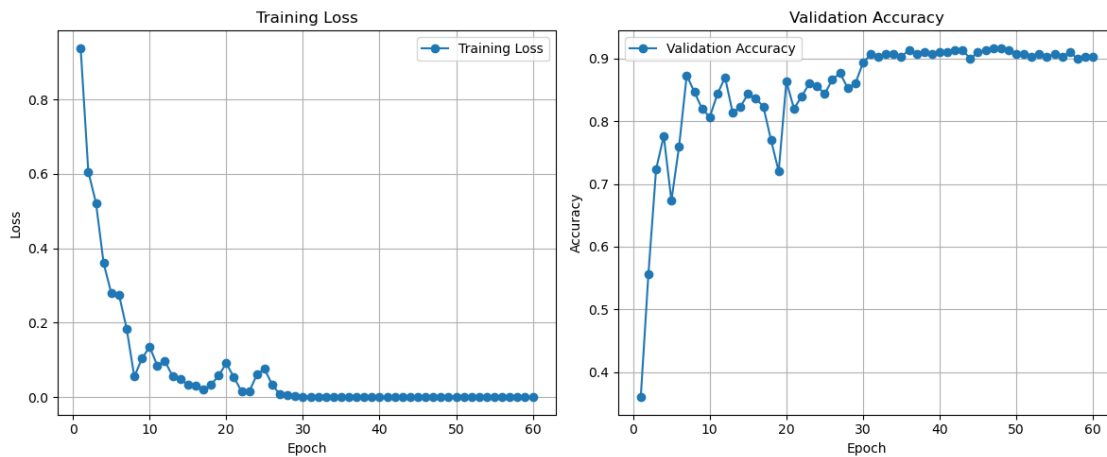
=== Training: VGG11BN_2c_with_dropout_weightdecay (Batch size: 32) ===

Training VGG11BN_2c_with_dropout_weightdecay_bs32_20250414_212952:

100%| | 2880/2880 [02:27<00:00, 19.48batch/s, epoch=60/60, phase=val,
loss=2.0090, acc=0.7500]

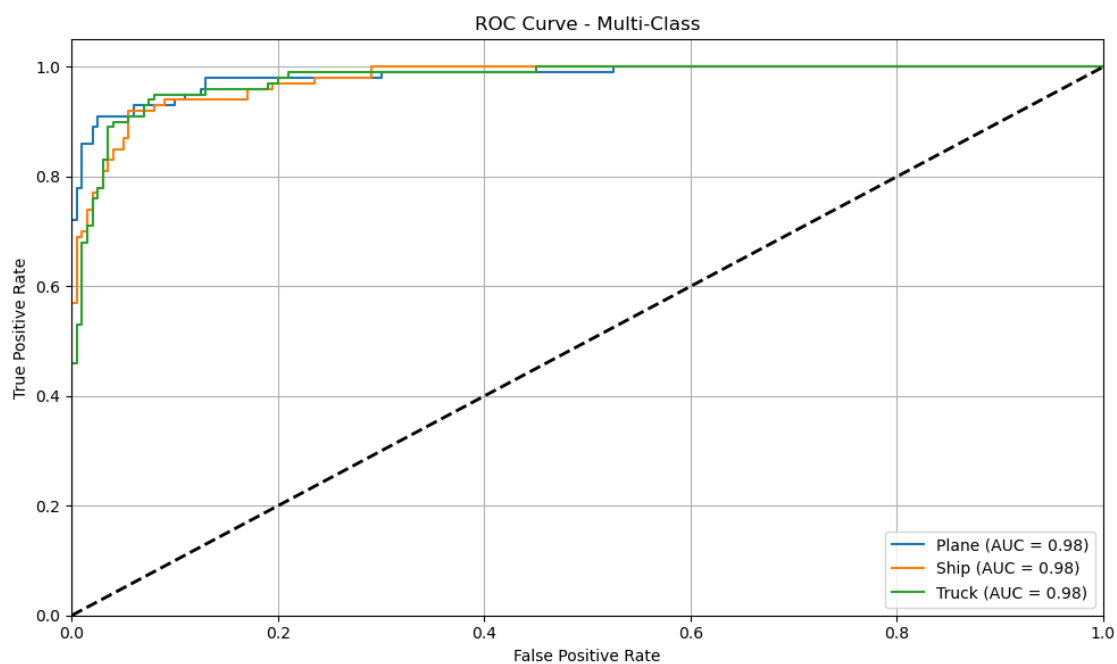
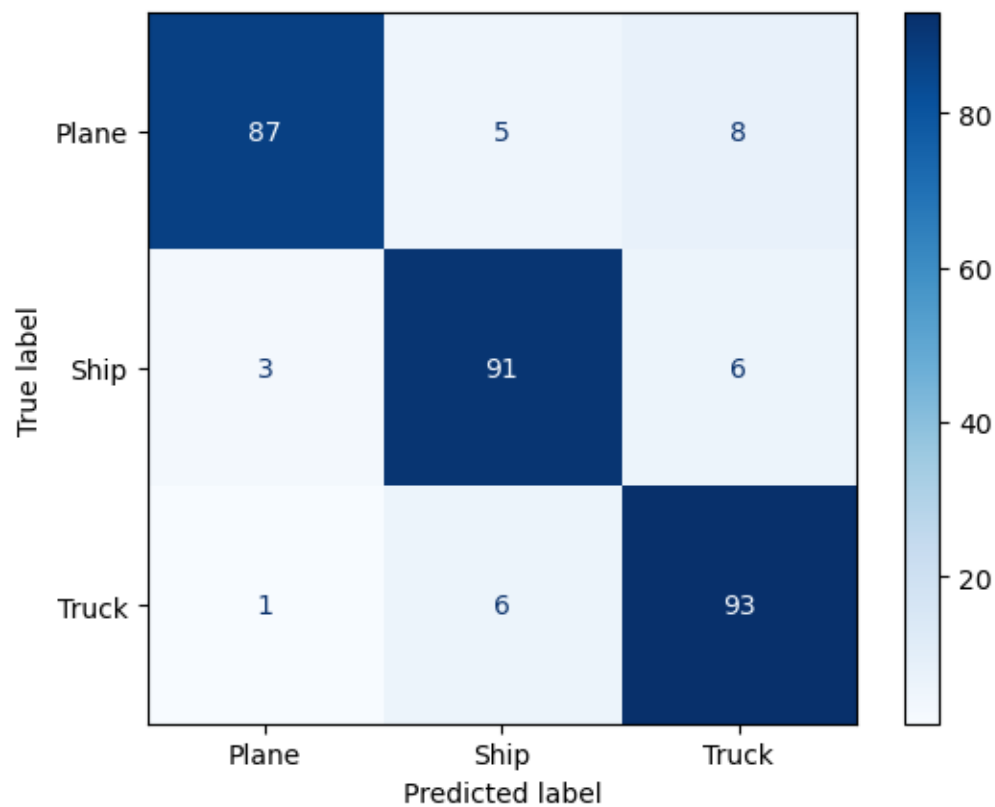
Best Val Accuracy: 0.9167

Total training time: 147.86s



	precision	recall	f1-score	support
Plane	0.96	0.87	0.91	100
Ship	0.89	0.91	0.90	100
Truck	0.87	0.93	0.90	100
accuracy			0.90	300
macro avg	0.91	0.90	0.90	300
weighted avg	0.91	0.90	0.90	300

```
+-----+
| Validation Loss: 0.5429, Validation Accuracy: 0.9033 |
+-----+
```



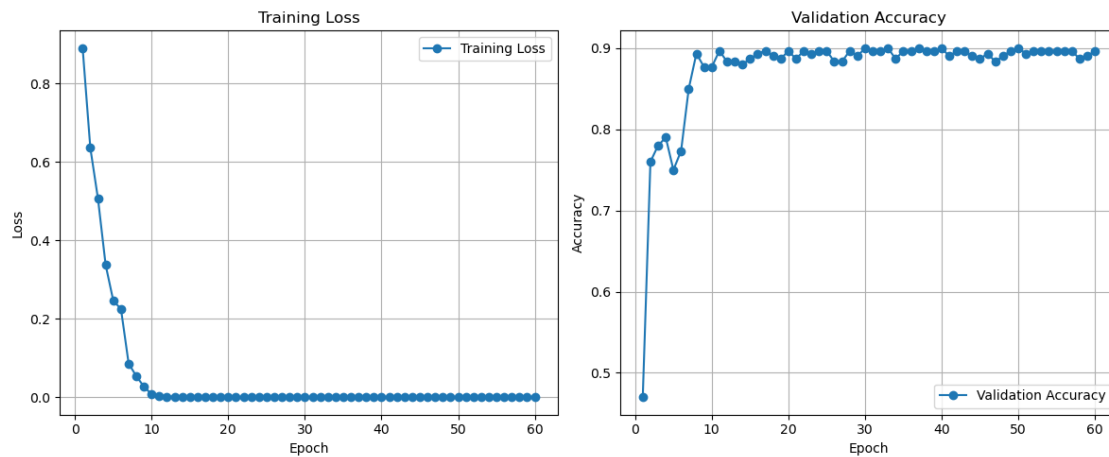
[ROC] Saved ROC plot to: plots/ROC_VGG11BN_2c_with_dropout_weightdecay_bs32.png

=== Training: VGG11BN_2c_with_dropout_weightdecay_scheduler (Batch size: 32) ===

Training VGG11BN_2c_with_dropout_weightdecay_scheduler_bs32_20250414_213222:
100%| | 2880/2880 [02:27<00:00, 19.50batch/s, epoch=60/60, phase=val,
loss=1.1999, acc=0.8333]

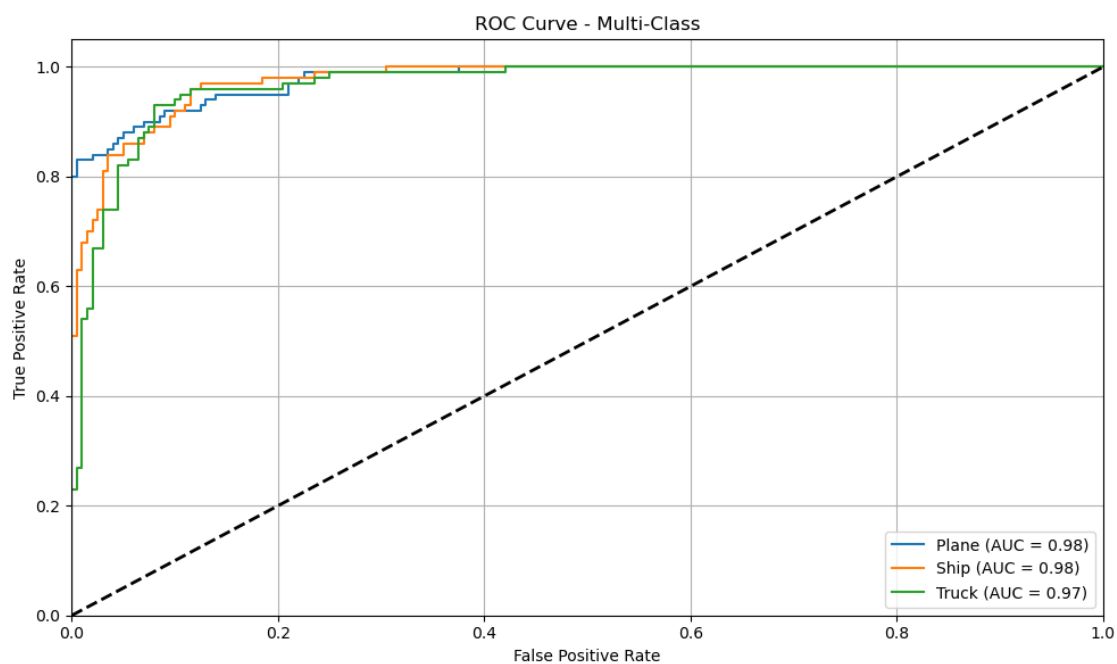
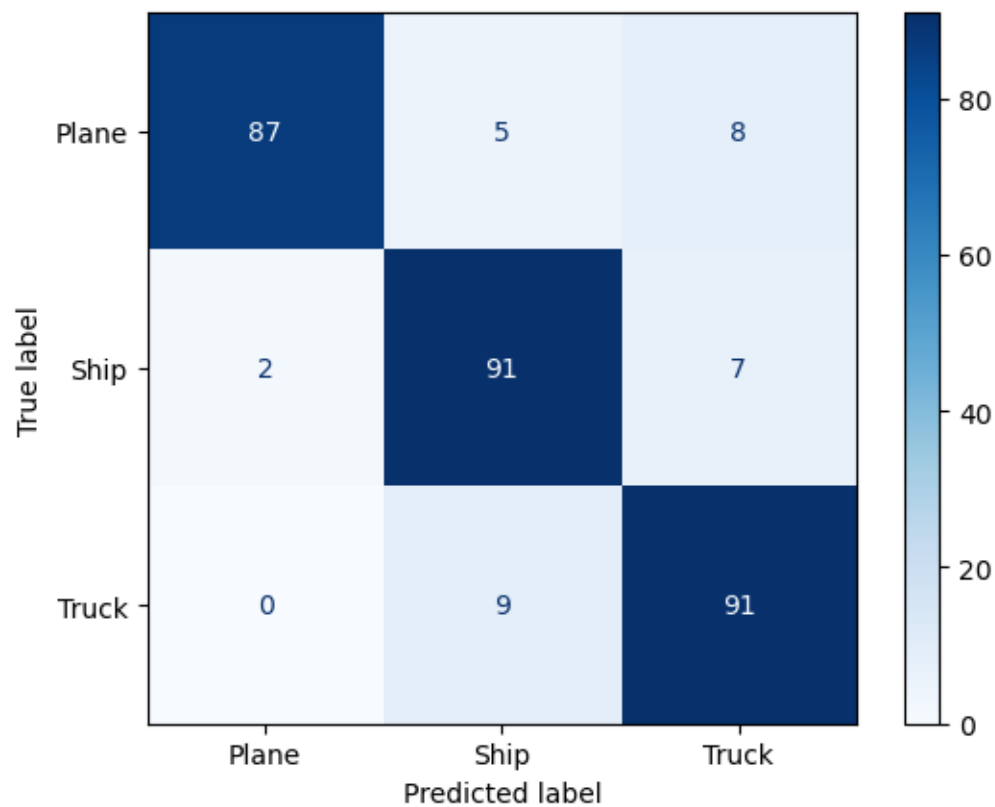
Best Val Accuracy: 0.9000

Total training time: 147.69s



	precision	recall	f1-score	support
Plane	0.98	0.87	0.92	100
Ship	0.87	0.91	0.89	100
Truck	0.86	0.91	0.88	100
accuracy			0.90	300
macro avg	0.90	0.90	0.90	300
weighted avg	0.90	0.90	0.90	300

```
+-----+
| Validation Loss: 0.4958, Validation Accuracy: 0.8967 |
+-----+
```



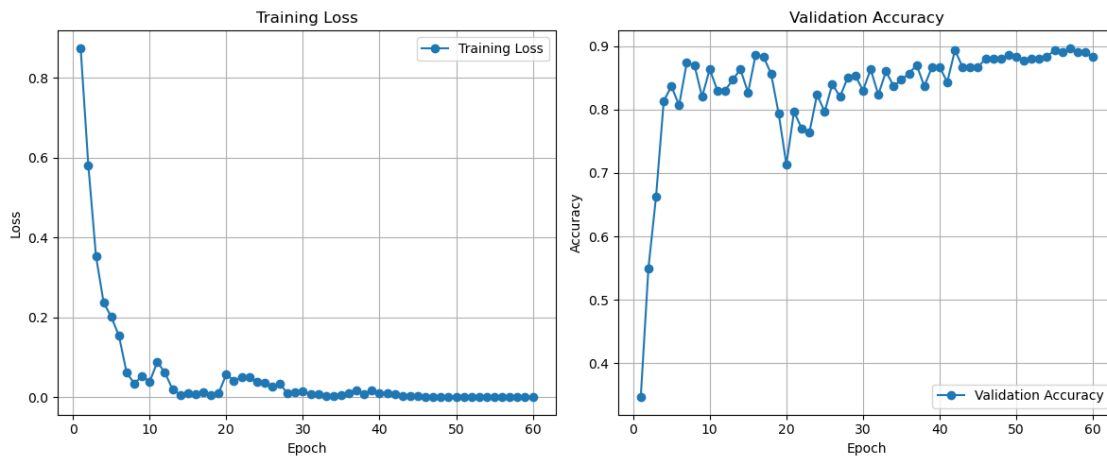
[ROC] Saved ROC plot to:
plots/ROC_VGG11BN_2c_with_dropout_weightdecay_scheduler_bs32.png

=== Training: VGG11BN_2a_no_dropout (Batch size: 64) ===

Training VGG11BN_2a_no_dropout_bs64_20250414_213452: 100%| | 1440/1440
[02:03<00:00, 11.65batch/s, epoch=60/60, phase=val, loss=0.8901, acc=0.8864]

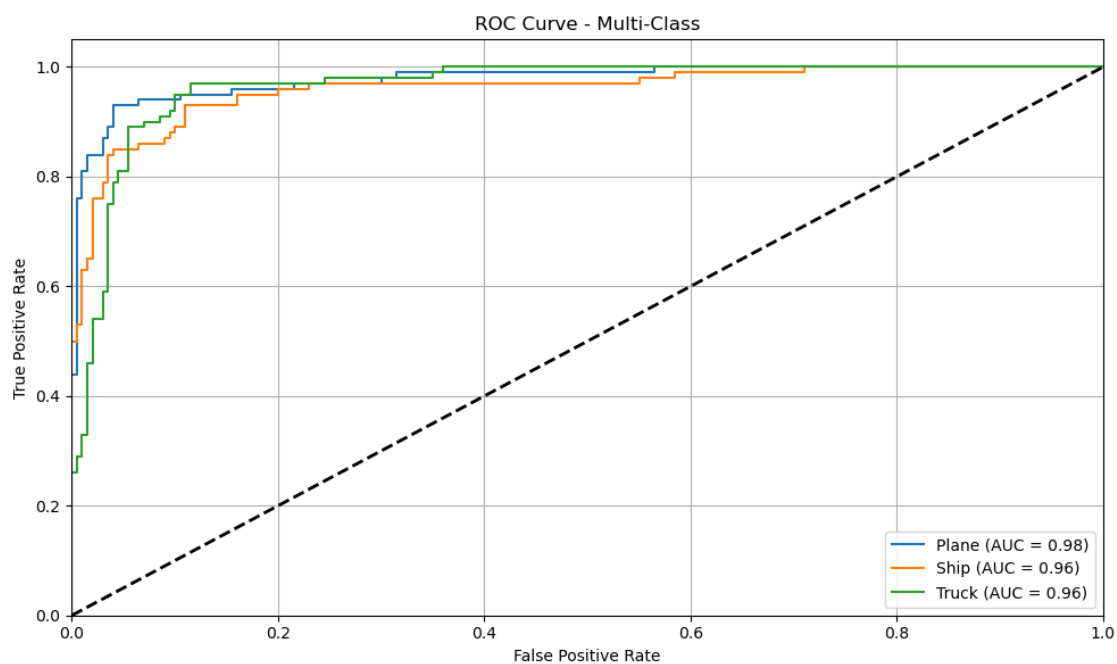
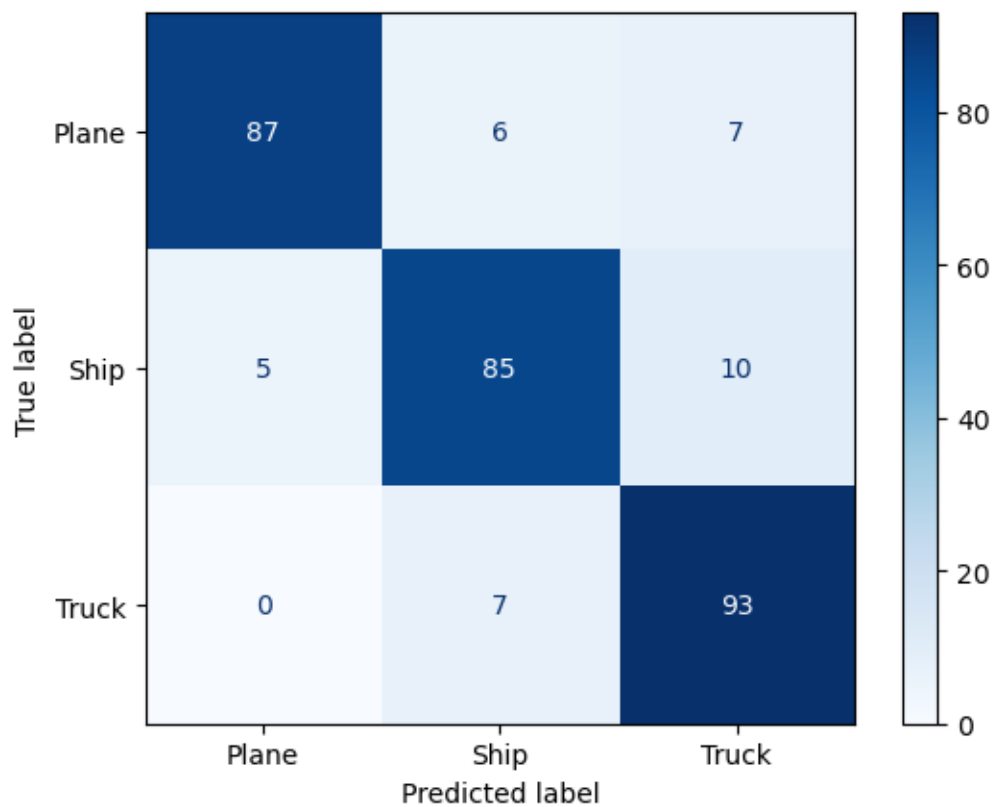
Best Val Accuracy: 0.8967

Total training time: 123.58s



	precision	recall	f1-score	support
Plane	0.95	0.87	0.91	100
Ship	0.87	0.85	0.86	100
Truck	0.85	0.93	0.89	100
accuracy			0.88	300
macro avg	0.89	0.88	0.88	300
weighted avg	0.89	0.88	0.88	300

```
+-----+
| Validation Loss: 0.8791, Validation Accuracy: 0.8833 |
+-----+
```



[ROC] Saved ROC plot to: plots/ROC_VGG11BN_2a_no_dropout_bs64.png

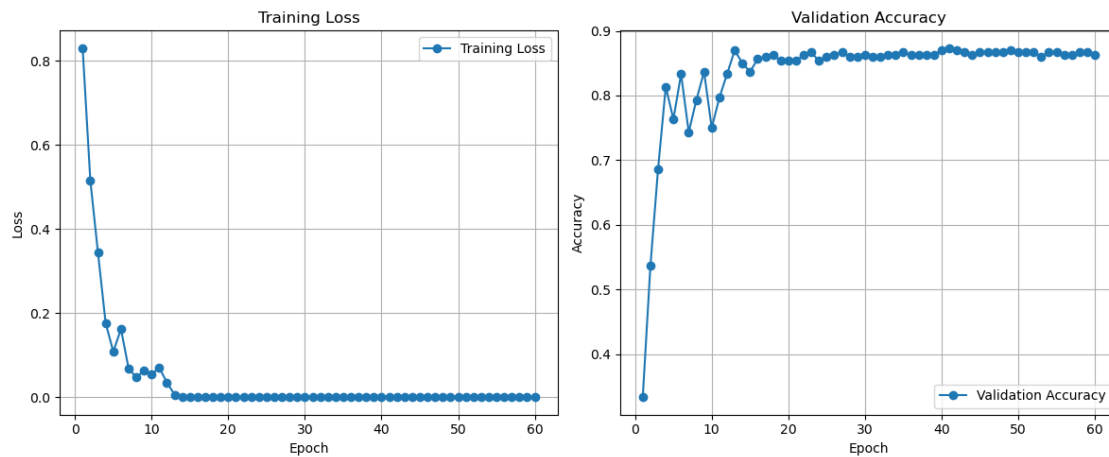
=== Training: VGG11BN_2a_no_dropout_weightdecay (Batch size: 64) ===

Training VGG11BN_2a_no_dropout_weightdecay_bs64_20250414_213657:

100%| | 1440/1440 [02:04<00:00, 11.52batch/s, epoch=60/60, phase=val,
loss=0.8532, acc=0.8182]

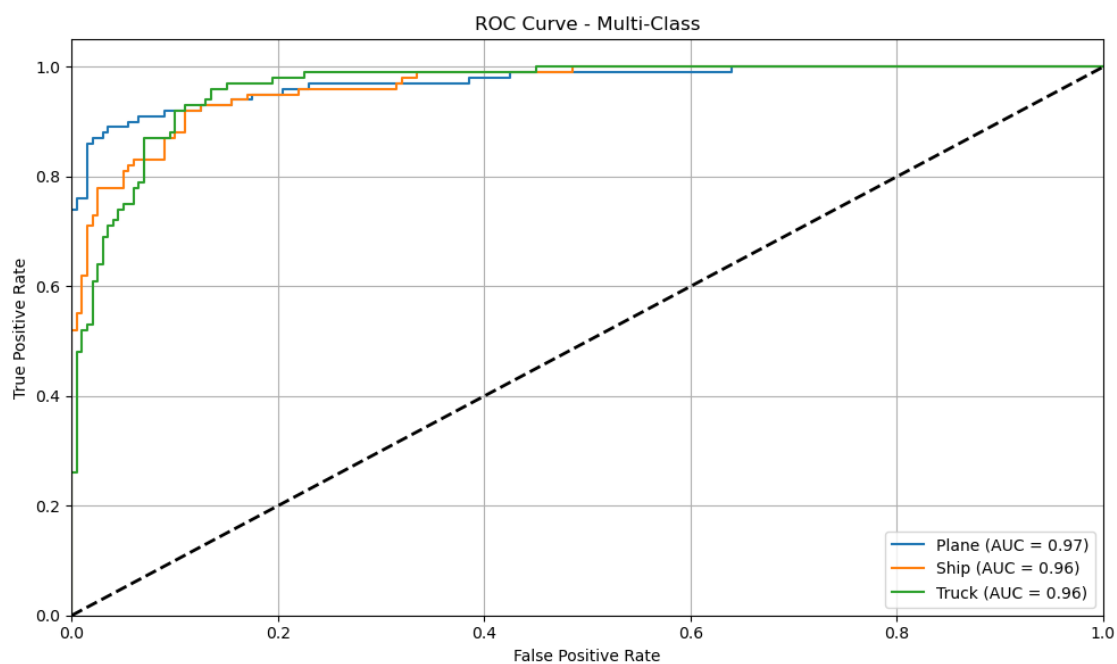
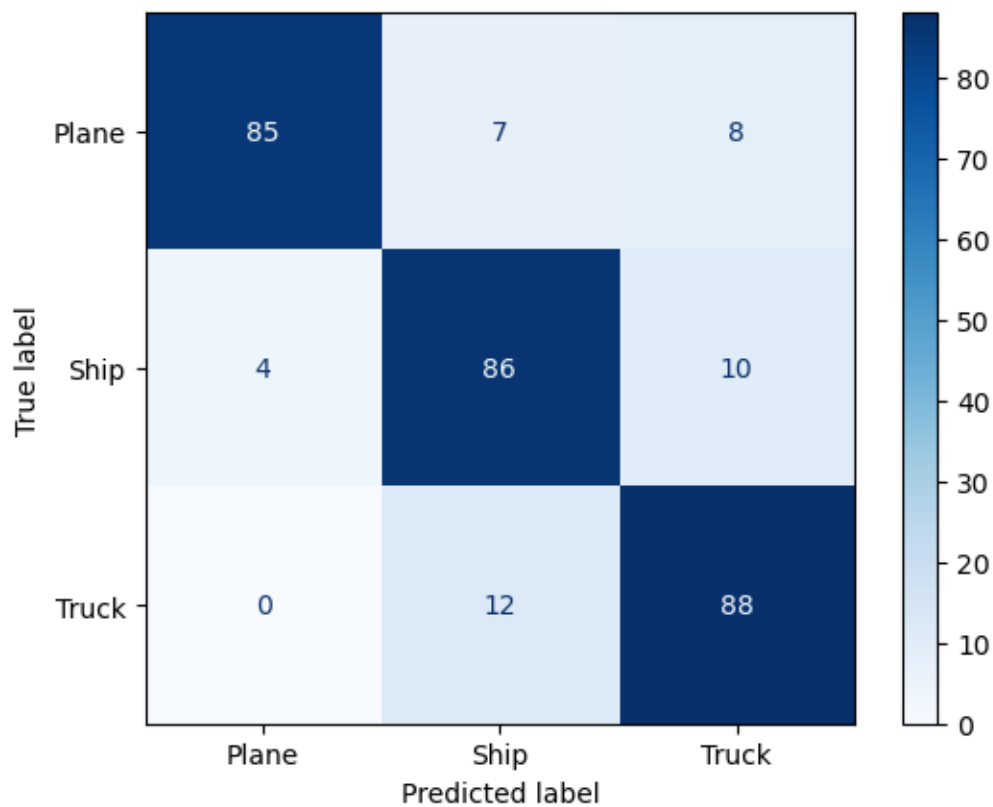
Best Val Accuracy: 0.8733

Total training time: 124.96s



	precision	recall	f1-score	support
Plane	0.96	0.85	0.90	100
Ship	0.82	0.86	0.84	100
Truck	0.83	0.88	0.85	100
accuracy			0.86	300
macro avg	0.87	0.86	0.86	300
weighted avg	0.87	0.86	0.86	300

```
+-----+
| Validation Loss: 0.6023, Validation Accuracy: 0.8633 |
+-----+
```



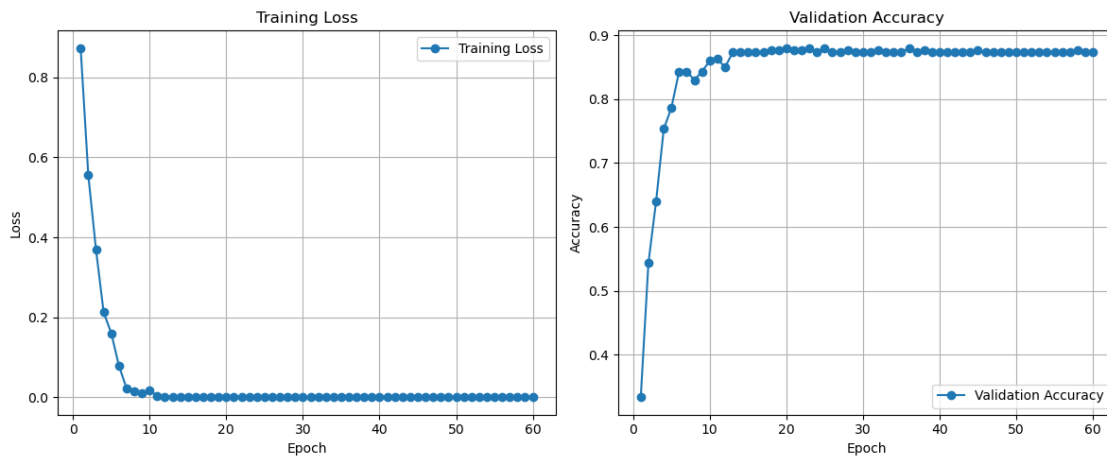
[ROC] Saved ROC plot to: plots/ROC_VGG11BN_2a_no_dropout_weightdecay_bs64.png

=== Training: VGG11BN_2a_no_dropout_weightdecay_scheduler (Batch size: 64) ===

Training VGG11BN_2a_no_dropout_weightdecay_scheduler_bs64_20250414_213904:
100%| | 1440/1440 [02:05<00:00, 11.51batch/s, epoch=60/60, phase=val,
loss=0.3533, acc=0.8864]

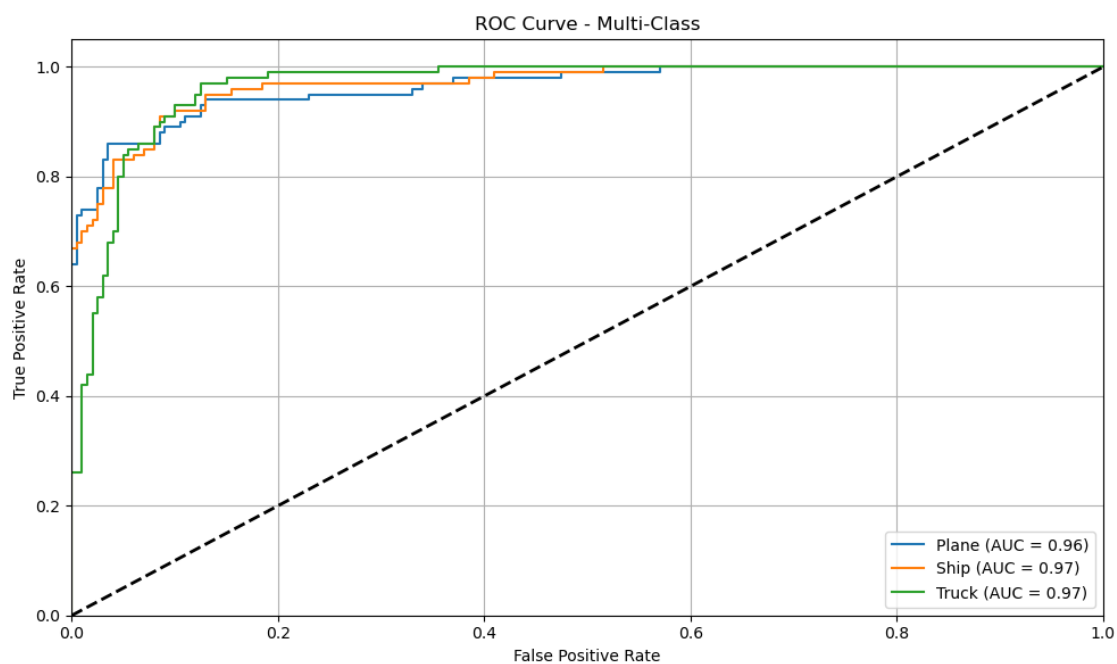
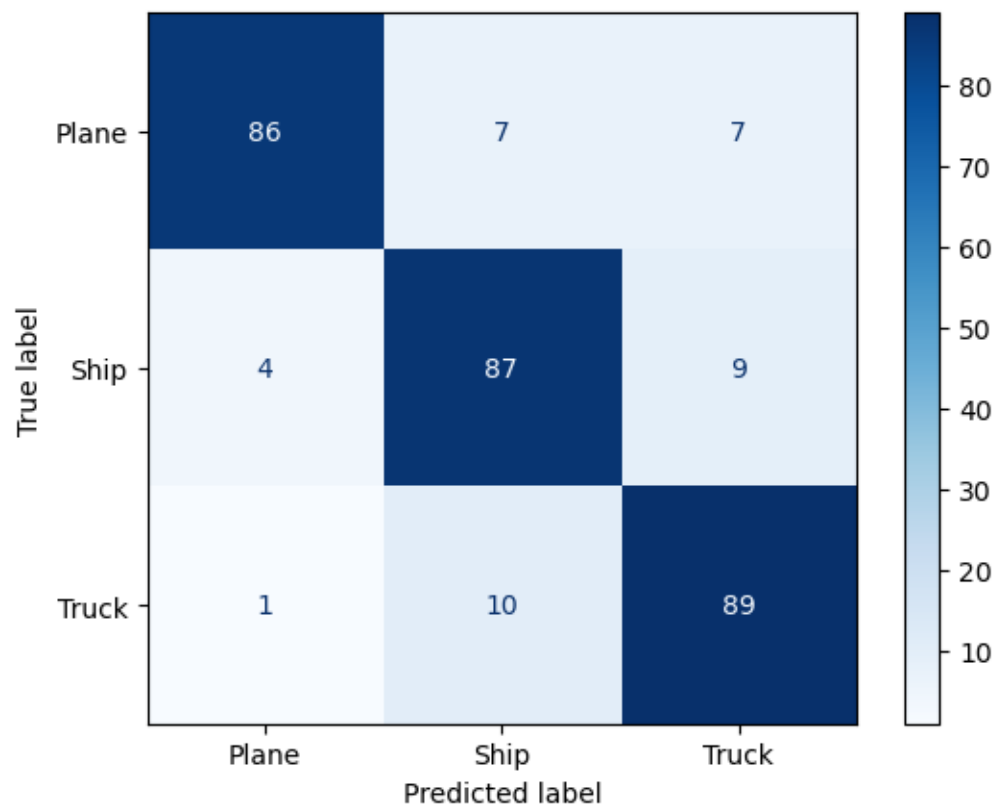
Best Val Accuracy: 0.8800

Total training time: 125.06s



	precision	recall	f1-score	support
Plane	0.95	0.86	0.90	100
Ship	0.84	0.87	0.85	100
Truck	0.85	0.89	0.87	100
accuracy			0.87	300
macro avg	0.88	0.87	0.87	300
weighted avg	0.88	0.87	0.87	300

```
+-----+
| Validation Loss: 0.5160, Validation Accuracy: 0.8733 |
+-----+
```



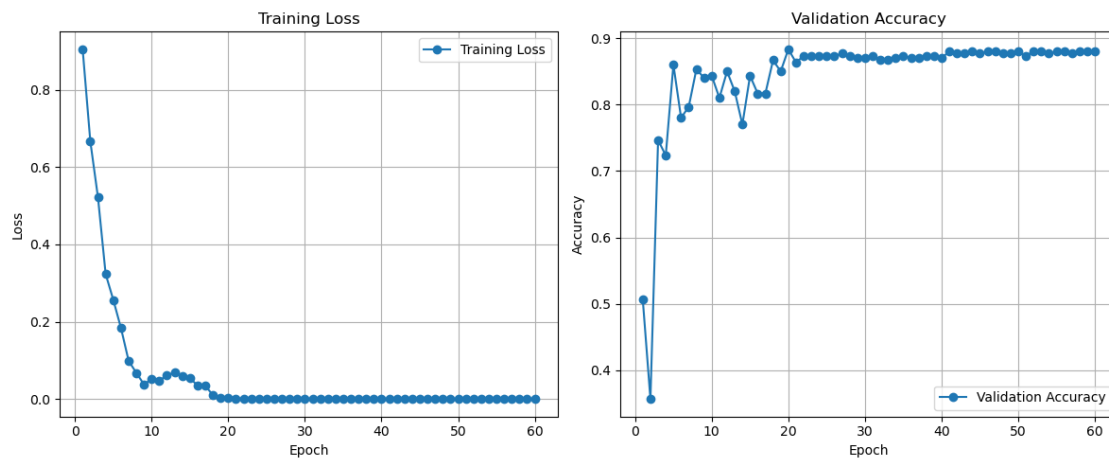
[ROC] Saved ROC plot to:
plots/ROC_VGG11BN_2a_no_dropout_weightdecay_scheduler_bs64.png

=== Training: VGG11BN_2c_with_dropout (Batch size: 64) ===

Training VGG11BN_2c_with_dropout_bs64_20250414_214111: 100%| |
1440/1440 [02:03<00:00, 11.67batch/s, epoch=60/60, phase=val, loss=1.0512,
acc=0.8409]

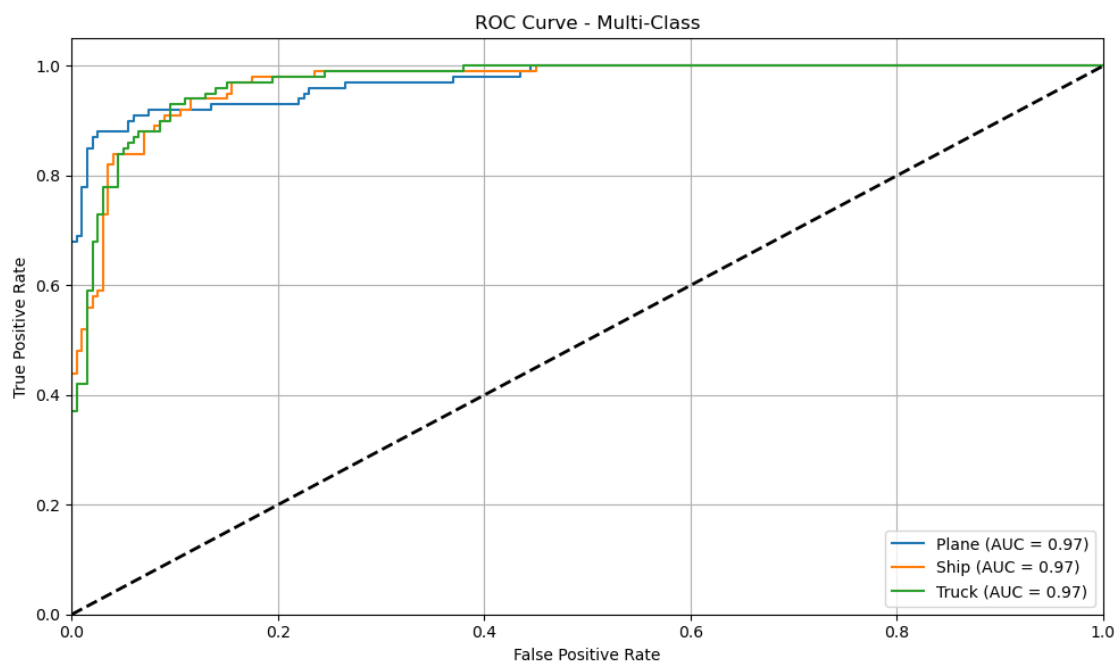
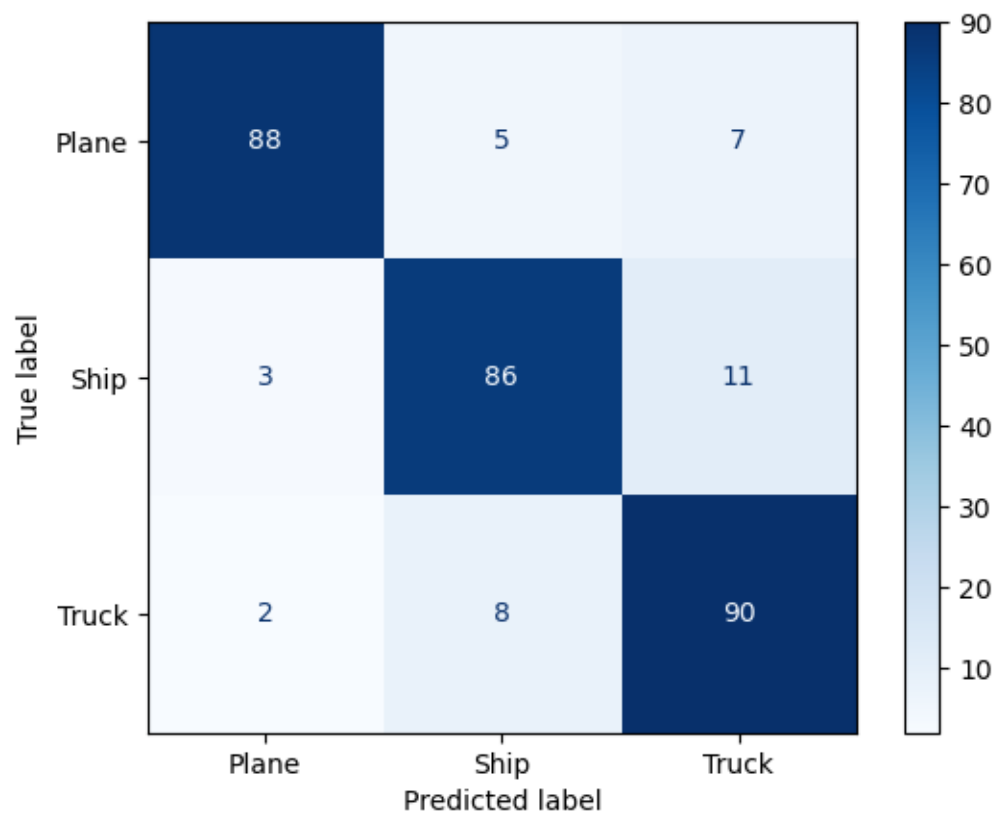
Best Val Accuracy: 0.8833

Total training time: 123.45s



	precision	recall	f1-score	support
Plane	0.95	0.88	0.91	100
Ship	0.87	0.86	0.86	100
Truck	0.83	0.90	0.87	100
accuracy			0.88	300
macro avg	0.88	0.88	0.88	300
weighted avg	0.88	0.88	0.88	300

```
+-----+
| Validation Loss: 0.7719, Validation Accuracy: 0.8800 |
+-----+
```



[ROC] Saved ROC plot to: plots/ROC_VGG11BN_2c_with_dropout_bs64.png

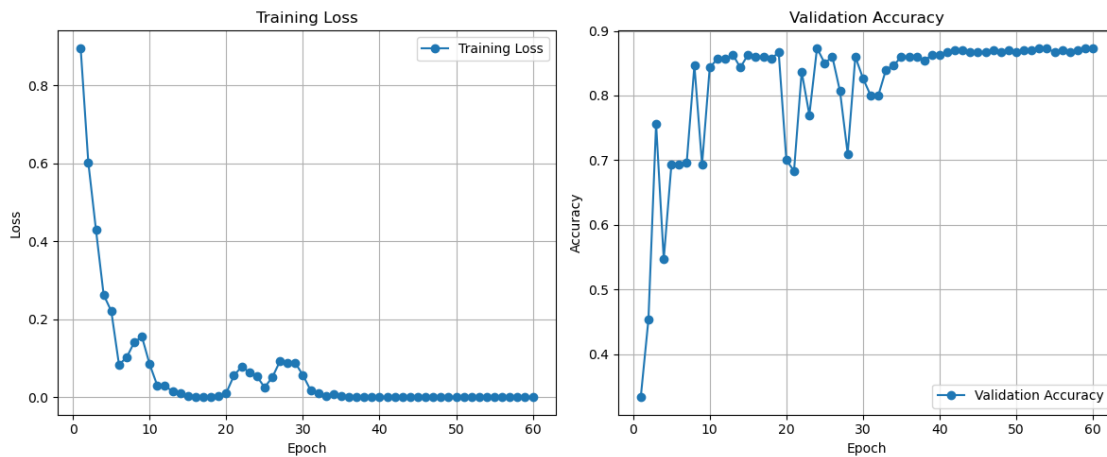
=== Training: VGG11BN_2c_with_dropout_weightdecay (Batch size: 64) ===

Training VGG11BN_2c_with_dropout_weightdecay_bs64_20250414_214317:

100%| | 1440/1440 [02:04<00:00, 11.53batch/s, epoch=60/60, phase=val,
loss=0.6904, acc=0.9091]

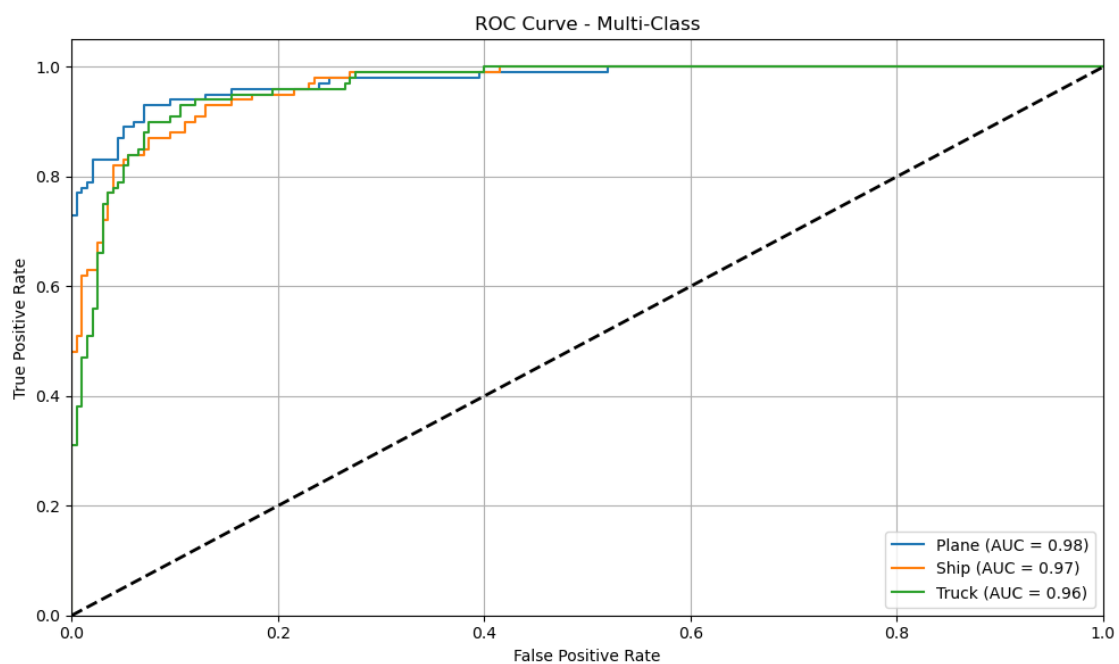
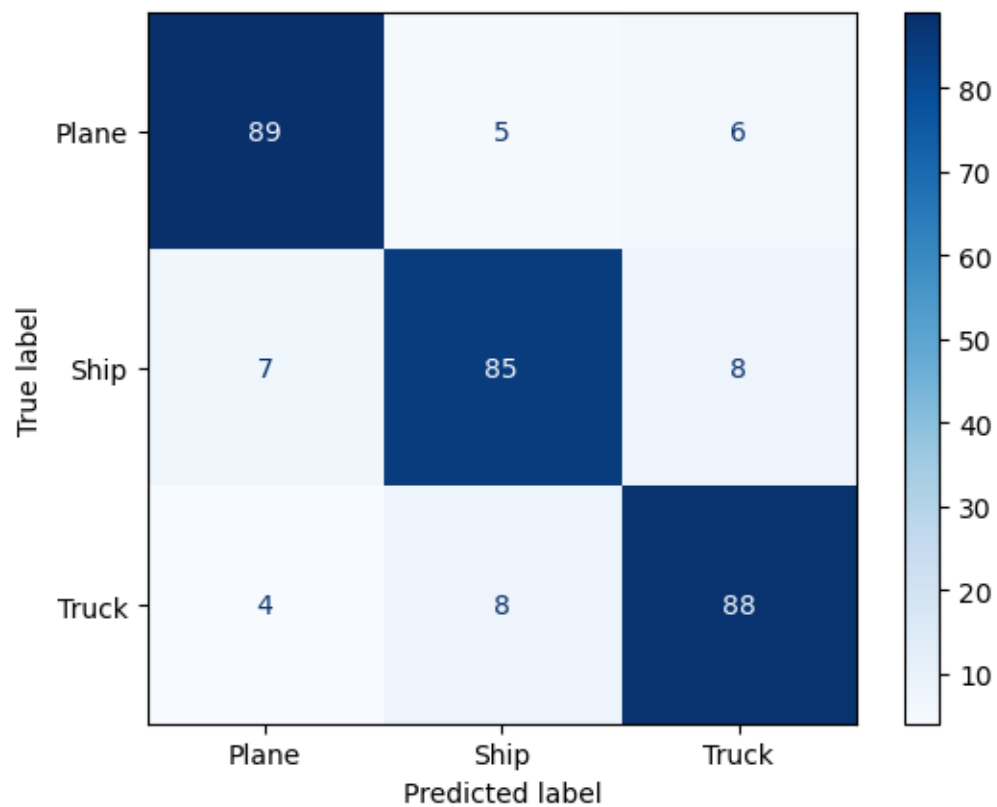
Best Val Accuracy: 0.8733

Total training time: 124.95s



	precision	recall	f1-score	support
Plane	0.89	0.89	0.89	100
Ship	0.87	0.85	0.86	100
Truck	0.86	0.88	0.87	100
accuracy			0.87	300
macro avg	0.87	0.87	0.87	300
weighted avg	0.87	0.87	0.87	300

```
+-----+
| Validation Loss: 0.7203, Validation Accuracy: 0.8733 |
+-----+
```



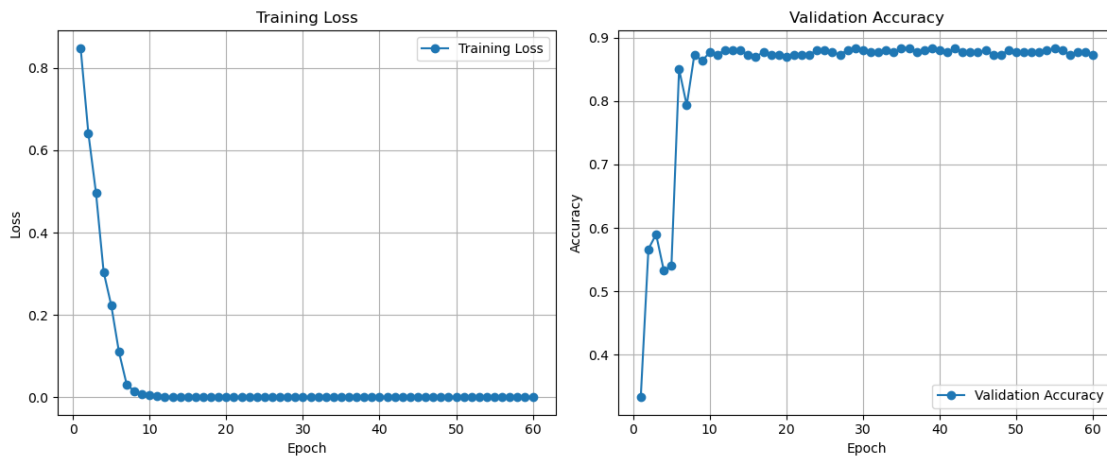
[ROC] Saved ROC plot to: plots/ROC_VGG11BN_2c_with_dropout_weightdecay_bs64.png

=== Training: VGG11BN_2c_with_dropout_weightdecay_scheduler (Batch size: 64) ===

Training VGG11BN_2c_with_dropout_weightdecay_scheduler_bs64_20250414_214524:
100%| | 1440/1440 [02:05<00:00, 11.52batch/s, epoch=60/60, phase=val,
loss=0.3770, acc=0.9318]

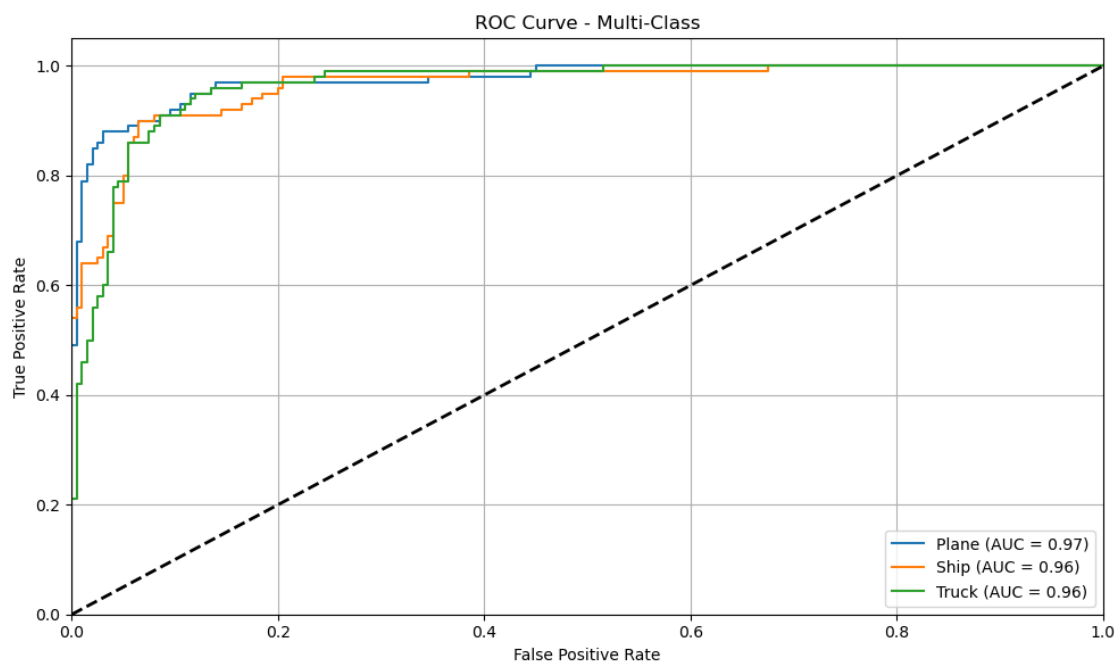
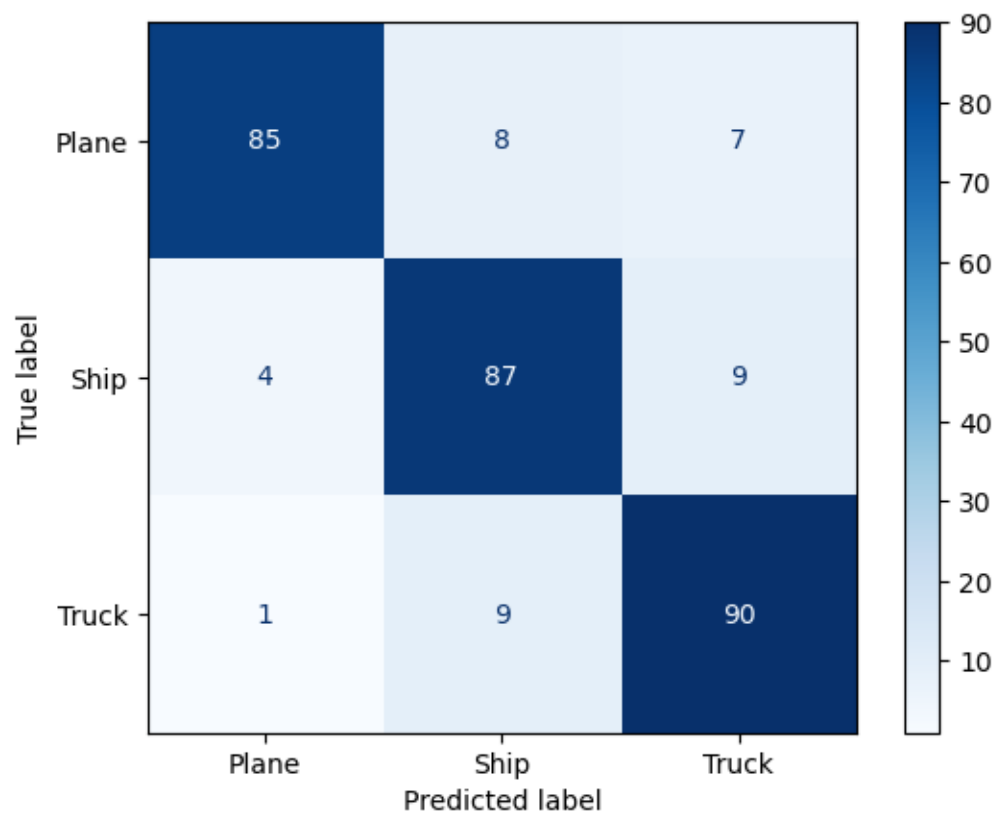
Best Val Accuracy: 0.8833

Total training time: 125.03s



	precision	recall	f1-score	support
Plane	0.94	0.85	0.89	100
Ship	0.84	0.87	0.85	100
Truck	0.85	0.90	0.87	100
accuracy			0.87	300
macro avg	0.88	0.87	0.87	300
weighted avg	0.88	0.87	0.87	300

```
+-----+
| Validation Loss: 0.5213, Validation Accuracy: 0.8733 |
+-----+
```



[ROC] Saved ROC plot to:
plots/ROC_VGG11BN_2c_with_dropout_weightdecay_scheduler_bs64.png

```
[ ]: from torch.utils.data import DataLoader
from src.utils import ImageDataset, set_seed, print_framed_metrics
from src.vgg11bn import VGG11BN
from src.train import train_model
from src.evaluation import classification_summary, plot_confusion_matrix,
    plot_multiclass_roc
import torch
import torch.nn as nn
import torch.optim as optim
from torch.optim.lr_scheduler import StepLR

# Set seed and device
set_seed(42)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# Prepare datasets once
train_dataset = ImageDataset(train_images, train_labels)
val_dataset = ImageDataset(val_images, val_labels)

# Define experiment configurations
configs = [
    {
        "name": "VGG11BN_with_dropout_Adam",
        "dropout": True,
        "weight_decay": 1e-4,
        "use_scheduler": False,
        "batch_size": 32,
        "optimizer": "adam", # choose "adam" or "sgd"
        "lr": 1e-4,
    },
    {
        "name": "VGG11BN_with_dropout_SGD",
        "dropout": True,
        "weight_decay": 1e-4,
        "use_scheduler": False,
        "batch_size": 32,
        "optimizer": "sgd",
        "lr": 1e-3,
    },
    # Add more configurations as needed.
]

# Run each experiment
for cfg in configs:
```

```

print(f"\n\n=== Training: {cfg['name']} (Batch size: {cfg['batch_size']})\n")

# Create dataloaders with specified batch size
dataloaders = {
    'train': DataLoader(
        train_dataset,
        batch_size=cfg['batch_size'],
        shuffle=True,
        pin_memory=True
    ),
    'val': DataLoader(
        val_dataset,
        batch_size=cfg['batch_size'],
        shuffle=False,
        pin_memory=True
    )
}

# Create model and loss function
model = VGG11BN(num_classes=3, dropout=cfg["dropout"]).to(device)
criterion = nn.CrossEntropyLoss()

# Setup optimizer based on configuration
optimizer_name = cfg.get("optimizer", "adam").lower()
lr = cfg.get("lr", 5e-5) # default lr for Adam; override in config if
needed
if optimizer_name == "sgd":
    optimizer = optim.SGD(model.parameters(), lr=lr,
weight_decay=cfg["weight_decay"],momentum=0.9)
else:
    optimizer = optim.Adam(model.parameters(), lr=lr,
weight_decay=cfg["weight_decay"])

# Setup scheduler (if any)
scheduler = StepLR(optimizer, step_size=15, gamma=0.5) if
cfg["use_scheduler"] else None

# Train the model
trained_model = train_model(
    model=model,
    dataloaders=dataloaders,
    criterion=criterion,
    optimizer=optimizer,
    scheduler=scheduler,
    device=device,
    num_epochs=60,

```

```

        model_name=f"{cfg['name']}_bs{cfg['batch_size']}"
    )

    # Evaluate the model
    val_loss, val_acc, y_true, y_pred = classification_summary(
        model=trained_model,
        dataloader=dataloaders["val"],
        criterion=criterion,
        device=device,
        class_names=['Plane', 'Ship', 'Truck']
    )

    print_framed_metrics(val_loss, val_acc)

    # Generate plots
    plot_confusion_matrix(
        trained_model,
        dataloaders['val'],
        device,
        class_names=['Plane', 'Ship', 'Truck'],
        save_path=f"plots/{cfg['name']}_bs{cfg['batch_size']}_confusion_matrix.
→png"
    )

    plot_multiclass_roc(
        model=trained_model,
        dataloader=dataloaders['val'],
        device=device,
        num_classes=3,
        class_names=['Plane', 'Ship', 'Truck'],
        model_name=f"{cfg['name']}_bs{cfg['batch_size']}"
    )

```

=== Training: VGG11BN_with_dropout_Adam (Batch size: 32) ===

Training VGG11BN_with_dropout_Adam_bs32_20250417_175341: 100%| |
960/960 [00:50<00:00, 18.91batch/s, epoch=20/20, phase=val, loss=2.4433,
acc=0.7500]

Best Val Accuracy: 0.8767

Total training time: 50.78s

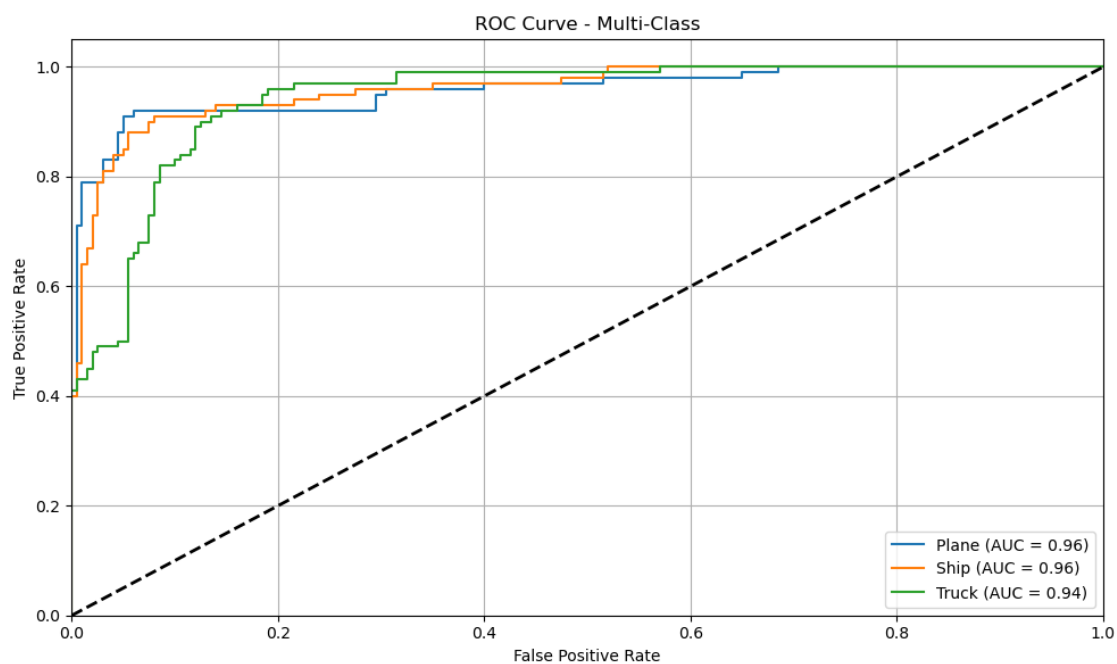
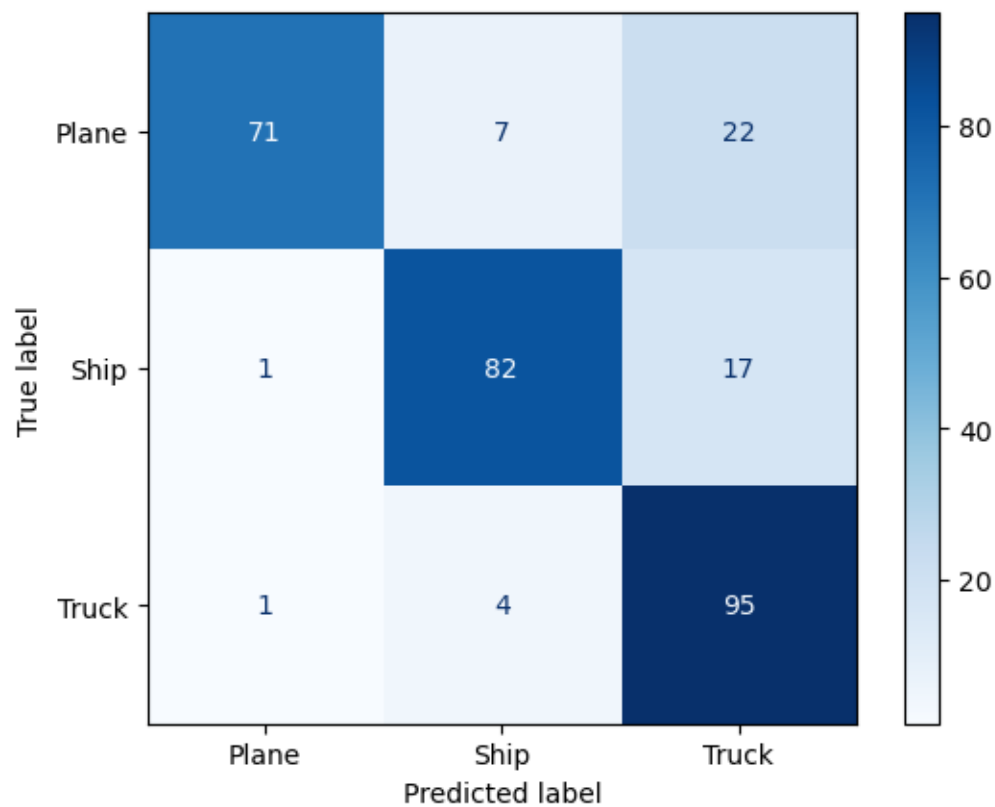


	precision	recall	f1-score	support
Plane	0.97	0.71	0.82	100
Ship	0.88	0.82	0.85	100
Truck	0.71	0.95	0.81	100
accuracy			0.83	300
macro avg	0.85	0.83	0.83	300
weighted avg	0.85	0.83	0.83	300

```

+-----+
| Validation Loss: 0.8805, Validation Accuracy: 0.8267 |
+-----+

```



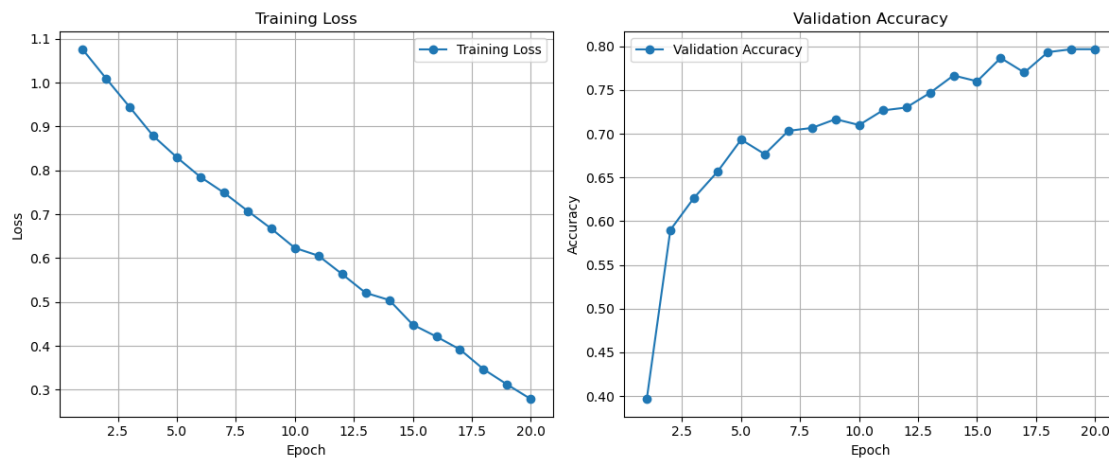
[ROC] Saved ROC plot to: plots/ROC_VGG11BN_with_dropout_Adam_bs32.png

=== Training: VGG11BN_with_dropout_SGD (Batch size: 32) ===

Training VGG11BN_with_dropout_SGD_bs32_20250417_175434: 100% | 960/960
[00:43<00:00, 21.88batch/s, epoch=20/20, phase=val, loss=0.4278, acc=0.7500]

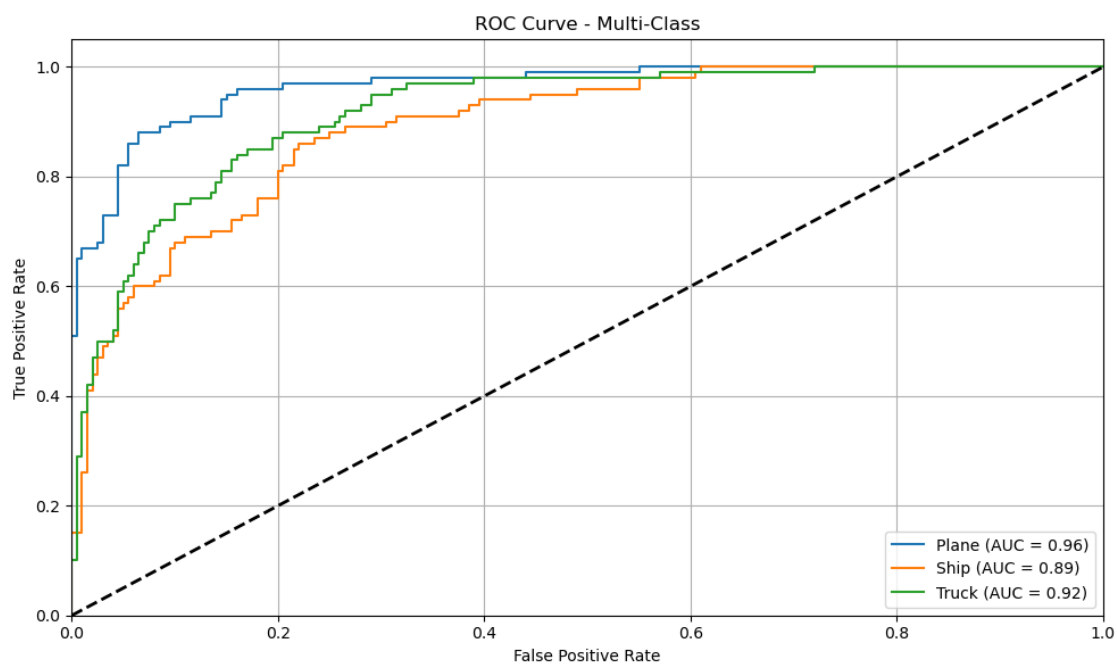
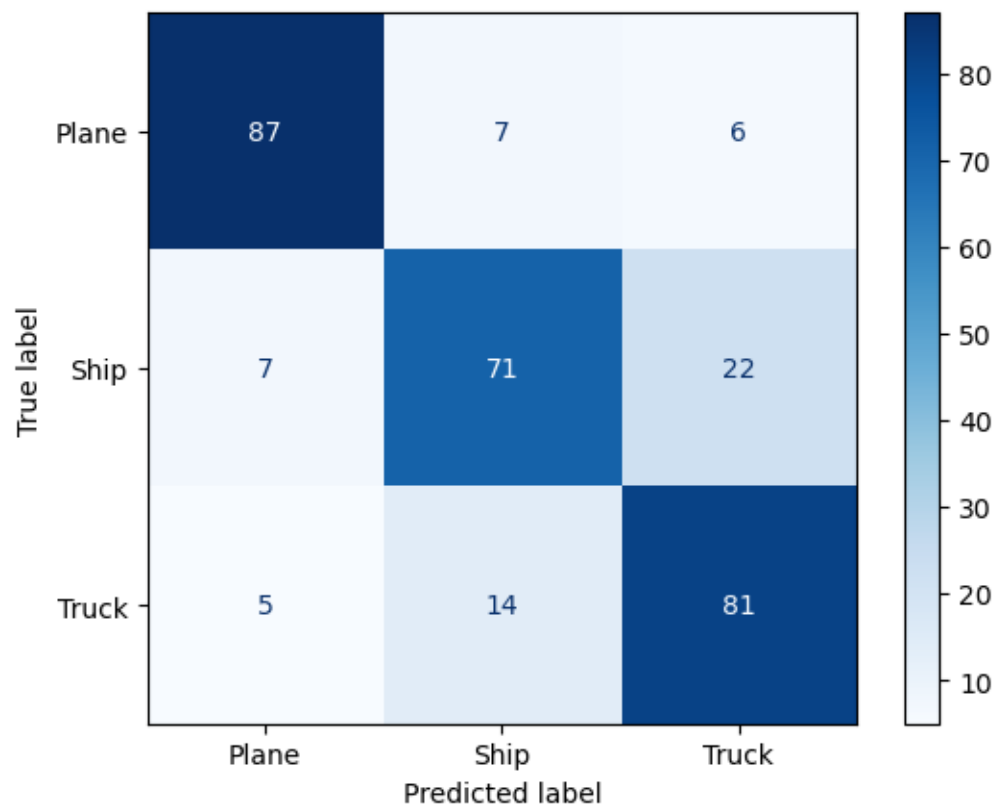
Best Val Accuracy: 0.7967

Total training time: 43.88s



	precision	recall	f1-score	support
Plane	0.88	0.87	0.87	100
Ship	0.77	0.71	0.74	100
Truck	0.74	0.81	0.78	100
accuracy			0.80	300
macro avg	0.80	0.80	0.80	300
weighted avg	0.80	0.80	0.80	300

```
+-----+
| Validation Loss: 0.4884, Validation Accuracy: 0.7967 |
+-----+
```



[ROC] Saved ROC plot to: plots/ROC_VGG11BN_with_dropout_SGD_bs32.png

[]:

